Rozdział 5

Wielka i szybka: Wykorzystanie hierarchii pamięci

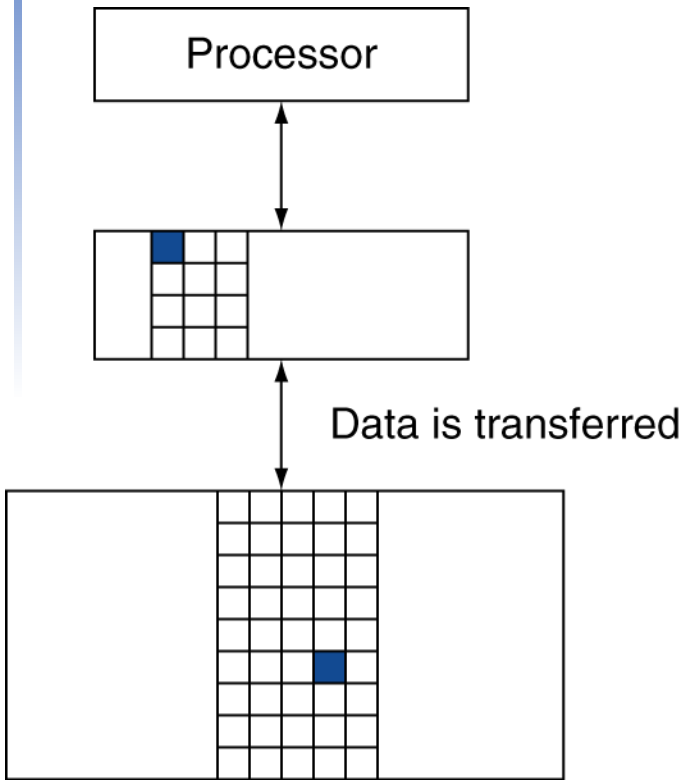
Zasada lokalności

- W danym momencie programy korzystają z małego obszaru przestrzeni adresowej
- Lokalność czasowa (Temporal locality)
  - Ostatnio adresowane dane prawdopodobnie zostaną wkrótce ponownie zaadresowane
  - np. instrukcje pętli, zmienne w indukcji
- Lokalność przestrzenna (Spatial locality)
  - Dane w pobliżu danych zaadresowanych też najprawdopodobniej będą użyte
  - np. kolejne instrukcje, elementy tablicy

Korzystanie z lokalności

- Hierarchia pamięci
- Chowaj wszystko na dysku
- Kopiuj ostatnio adresowane (i pobliskie) dane z dysku do (mniejszej) pamięci DRAM
  - Pamięć główna (Main memory)
- Kopiuj ostatnio adresowane (i pobliskie) dane z DRAM do mniejszej pamięci SRAM
  - Pamięć podręczna (Cache memory) przy CPU

Poziomy hierarchii pamięci



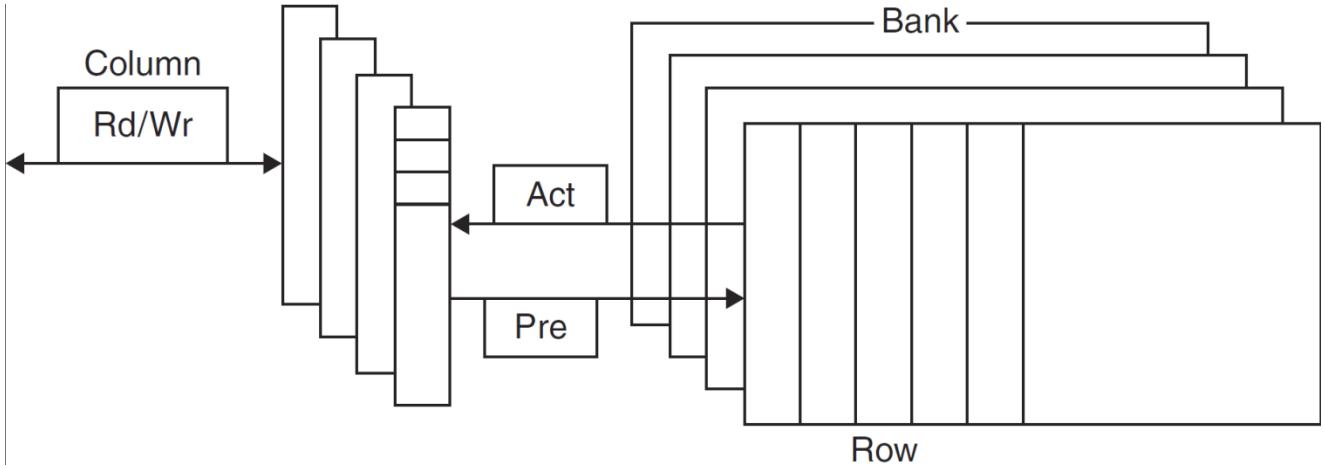
- Blok (linia): jednostka kopiowania
  - Może to być wiele słów
- Jeśli adresowane dane są dostępne w poziomie wyższym
  - Trafienie (Hit): dostęp w tym poziomie
    - Wskaźnik trafień (Hit ratio): stosunek trafienia/adresowania
- Jeśli nie są dostępne
  - Nietrafienie (Miss): blok trzeba kopiować z niższego poziomu
    - Czas: koszt nietrafienia (miss penalty)
    - Wskaźnik nietrafień (Miss ratio): nietrafienia/adresowania = 1 – hit ratio
  - Dane osiągane z wyższego poziomu

Technologia pamięci

- Static (statyczna) RAM (SRAM)
  - 0.5ns – 2.5ns, $2000 – $5000 per GB
- Dynamic (dynamiczna) RAM (DRAM)
  - 50ns – 70ns, $20 – $75 per GB
- Dysk magnetyczny
  - 5ms – 20ms, $0.20 – $2 per GB
- Ideał pamięci
  - Czas dostępu jak dla SRAM
  - Pojemność i koszt/GB jak dla dysku

Technologia DRAM

- Dane jako ładunek kondensatora
  - Dostęp przez pojedynczy tranzystor
  - Musi być okresowo odświeżany
    - Odczyt ładunku, po czym zapis
    - Wykonywany na „wierszu” DRAM

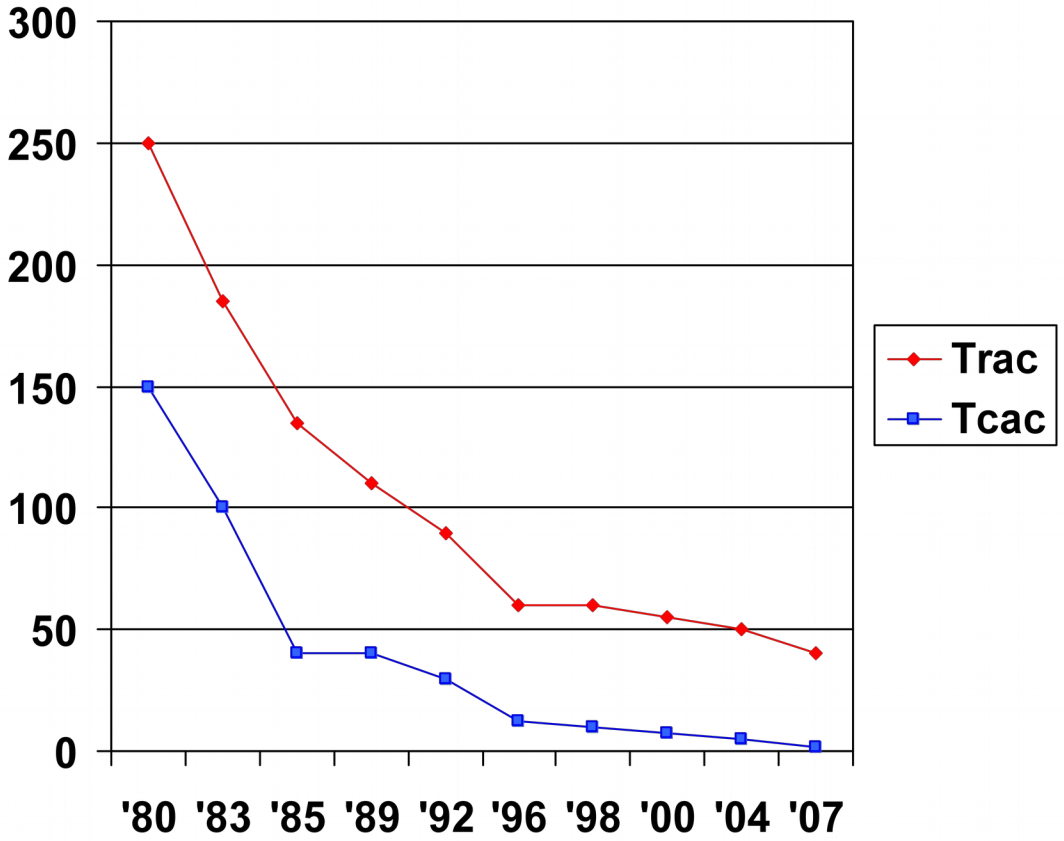


Zaawansowana DRAM

- Bity DRAM są ułożone w prostokątną macierz
  - Dostęp do całego wiersza DRAM
  - Tryb seryjny (Burst mode): kolejne słowa wiersza uzyskiwane w krótszych odstępach
- Double data rate (DDR) DRAM
  - Transfer na przednim i tylnym zboczu zegara
- Quad data rate (QDR) DRAM
  - Niezależne wejścia i wyjścia DDR

Generacje DRAM

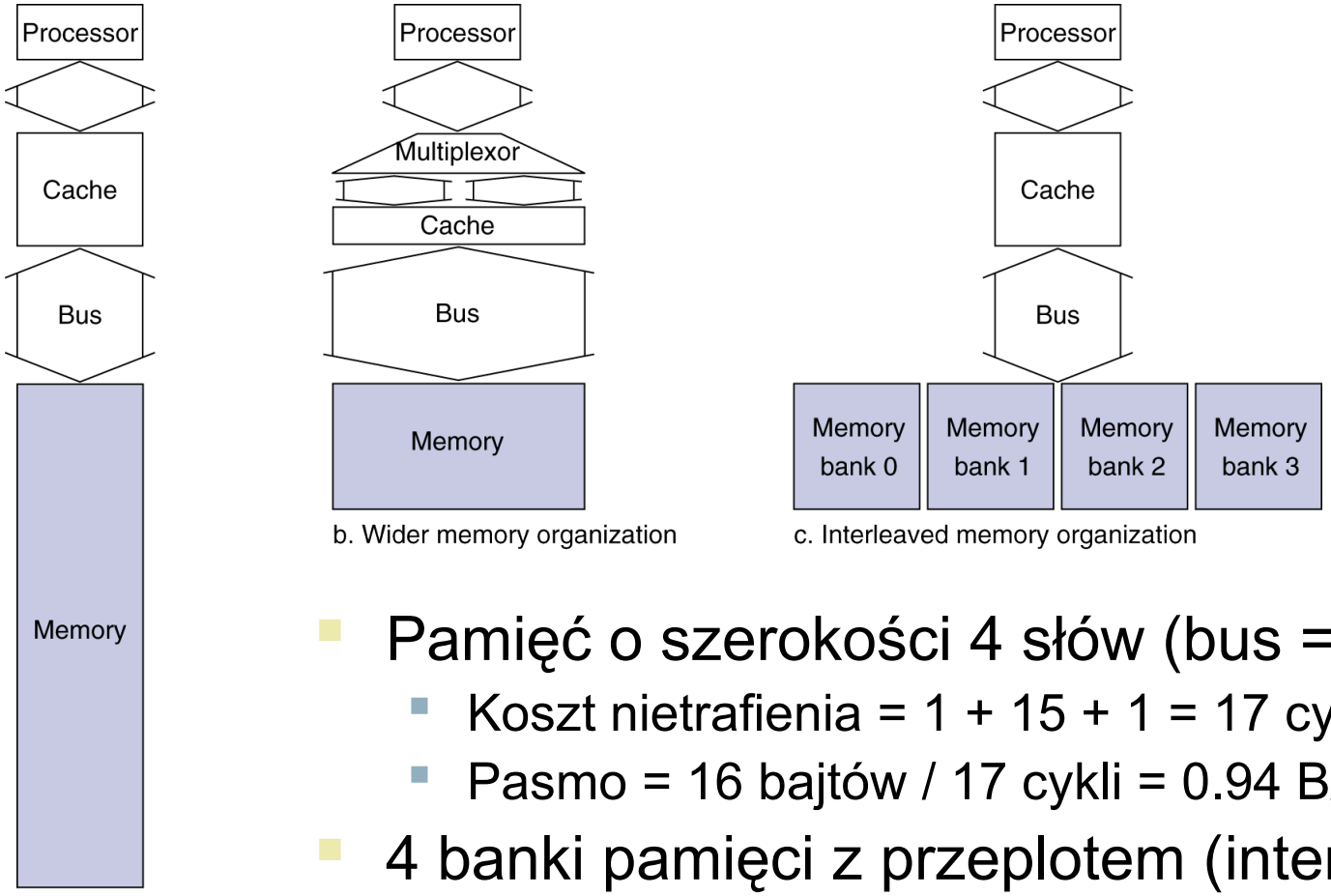
Rok	Pojemność	$/GB
1980	64Kbit	$1500000
1983	256Kbit	$500000
1985	1Mbit	$200000
1989	4Mbit	$50000
1992	16Mbit	$15000
1996	64Mbit	$10000
1998	128Mbit	$4000
2000	256Mbit	$1000
2004	512Mbit	$250
2007	1Gbit	$50



Przyspieszanie DRAM

- Bufor wiersza (Row buffer)
  - Pozwala na równoległy odczyt i odświeżanie wielu słów
- Synchroniczna DRAM
  - Pozwala na kolejne dostępy w serii bez podawania za każdym razem adresu
  - Zwiększa pasmo (bandwidth)
- Banki DRAM
  - Pozwala na równoczesny dostęp do wielu DRAM
  - Zwiększa pasmo

Zwiększanie pasma pamięci

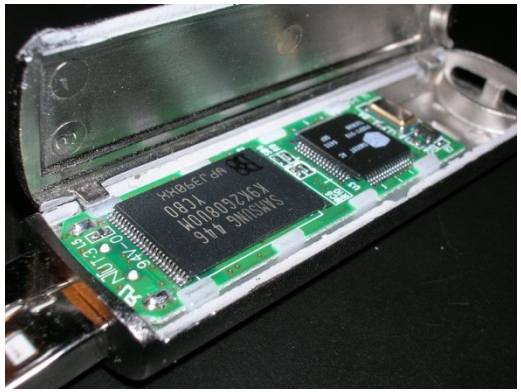


- Pamięć o szerokości 4 słów (bus = szyna)
  - Koszt nietrafienia = 1 + 15 + 1 = 17 cykli szyny
  - Pasmo = 16 bajtów / 17 cykli = 0.94 B/cykl
- 4 banki pamięci z przeplotem (interleaved)
  - Koszt nietrafienia = 1 + 15 + 4×1 = 20 cykli szyny
  - Pasmo = 16 bajtów / 20 cykli = 0.8 B/cykl

Pamięć Flash

- Nieulotna pamięć półprzewodnikowa
  - 100× – 1000× szybsza od dysku
  - Mniejsza, mniejszy pobór mocy, odporniejsza
  - Ale więcej $/GB (pomiędzy dyskiem i DRAM)





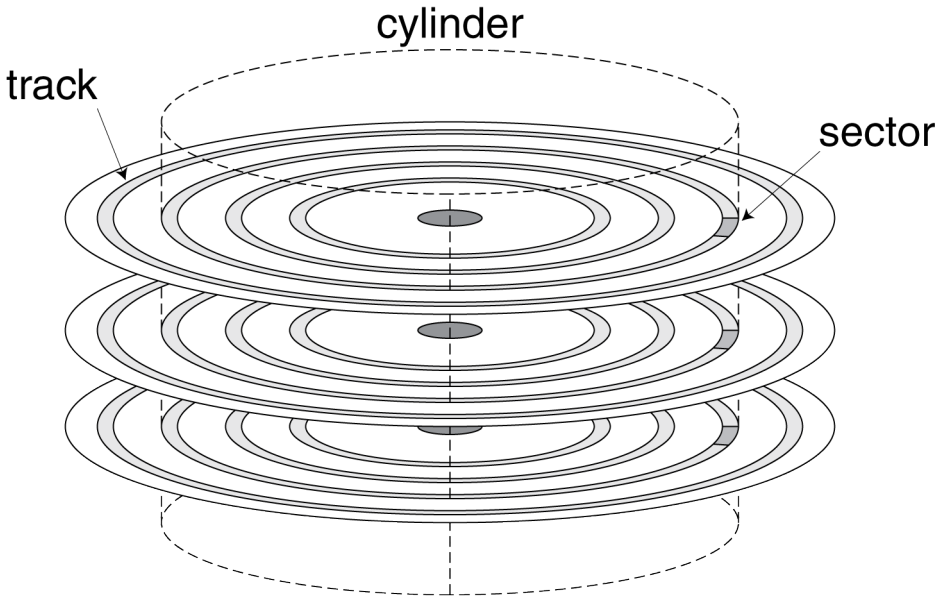
Rodzaje pamięci flash

- NOR flash: komórka bitowa jak w bramce NOR
  - Swobodny (random) dostęp przy odczycie/zapisie
  - Używana jako pamięć instrukcji w systemach zagnieżdżonych
- NAND flash: komórka bitowa jak w NAND
  - Gęstsza (bity/powierzchnię), ale dostęp blokami
  - Tańsza na GB
  - Używana do kluczy USB, pamięci multimedialnej, …
- Bity flash zużywają się po tysiącach dostępów
  - Nie nadaje się wprost w miejsce RAM lub dysków
  - Wyrównanie zużycia: przesuń dane do mniej zużytych bloków

Pamięć dyskowa

- Nieulotna rotująca pamięć magnetyczna





Sektory dysku i dostęp

- Każdy sektor zapamiętuje
  - ID sektora
  - Dane (512 bajtów, proponowane 4096 bajtów)
  - Kod korekcyjny (Error correcting code, ECC)
    - Używany do ukrywania defektów i błędów zapisu
  - Pola i przerwy synchronizacyjne
- Na dostęp do sektora składa się
  - Opóźnienie kolejki jeśli trwają inne dostępy
  - Szukanie: przesuwanie głowic
  - Opóźnienie wynikające z rotacji
  - Transfer danych
  - Opóźnienie wprowadzane przez sterownik

Przykład dostępu do dysku

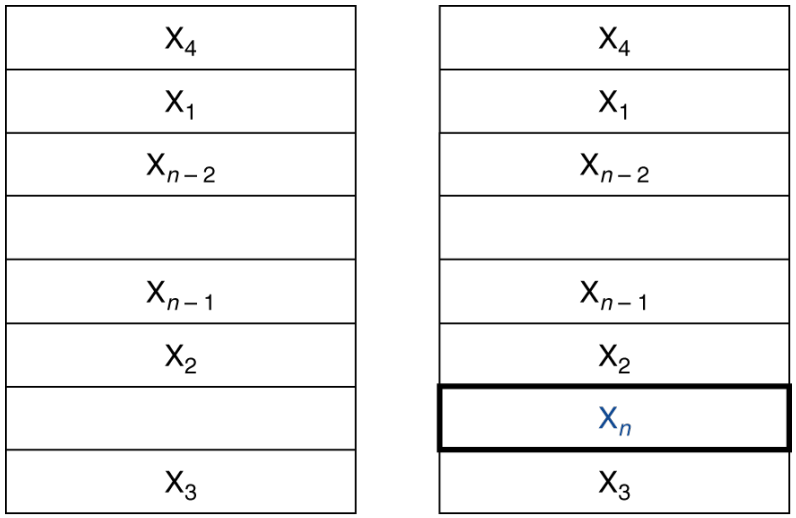
- Dane
  - 512B na sektor, 15,000rpm (obrotów na minutę), średni czas szukania 4ms, Szybkość transferu 100MB/s, opóźnienie sterownika 0.2ms, dysk bezczynny
- Średni czas odczytu
  - Czas szukania 4ms
    + ½ / (15,000/60) = 2ms opóźnienia rotacji
    + 512 / 100MB/s = 0.005ms czas transferu
    + 0.2ms opóźnienie sterownika
    = 6.2ms
- Jeśli faktyczny średni czas szukania 1ms
  - Średni czas odczytu = 3.2ms

Wydajność dysków

- Producenci podają średni czas szukania
  - Biorąc pod uwagę wszystkie możliwe przypadki
  - Lokalność i planowanie przez OS prowadzi do mniejszych faktycznych średnich czasów szukania
- Inteligentny sterownik dysków (smart disk controller) przypisując fizyczne sektory dysku
  - Prezentuje logiczny interfejs sektorów procesorowi
  - SCSI, ATA, SATA
- Napędy dysków zawierają pamięci podręczne
  - Sektory są ładowane w przewidywaniu ich użycia
  - Unika się opóźnień szukania i rotacji

Pamięć podręczna (Cache)

- Pamięć podręczna
  - Poziom w hierarchii pamięci najbliższy CPU
- Załóżmy, że zaadresowano X_1, …, X_{n-1}, X_n

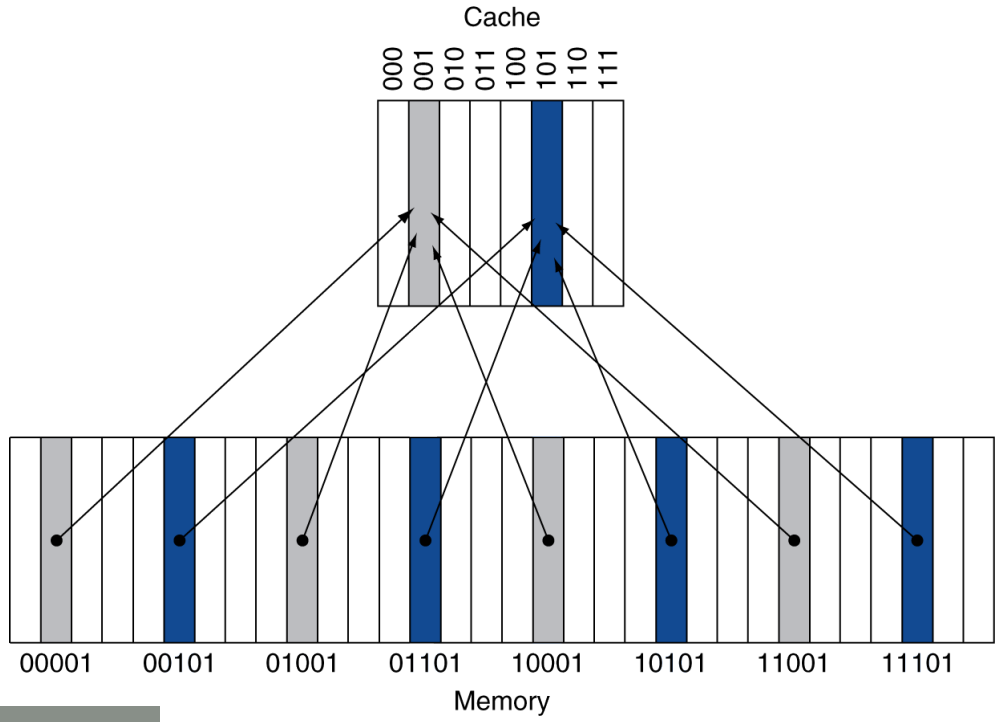


a. Before the reference to X_n b. After the reference to X_n

- Skąd wiemy, że dane są dostępne?
- Gdzie patrzymy?

Odwzorowanie bezpośrednie

- Położenie określone adresem
- Odwzorowane bezpośrednio: jedna lokacja
  - (Adres bloku) modulo (#Bloków w pamięci)



- #Bloków jest potęgą 2
- Użyj dolnych bitów adresu

Tagi i bity walidacji (valid bits)

- Skąd wiemy który blok jest zapamiętany w pamięci podręcznej?
  - Zapamiętaj także adres bloku
  - W rzeczywistości potrzebne tylko górne bity
  - Nazywane tagiem (tag)
- Co jeśli nie ma danych w danym miejscu?
  - Bit walidacji: 1 = są dane, 0 = nie ma danych
  - Początkowo 0

Przykład pamięci podręcznej

- 8-blokowa, 1 słowo/blok, odwzorowana bezpośrednio (directly mapped)
- Stan początkowy

Indeks	V	Tag	Dane
000	N		
001	N		
010	N		
011	N		
100	N		
101	N		
110	N		
111	N		

Przykład pamięci podręcznej

Adres słowa	Adres binarny	Hit/miss	Blok pamięci
22	10 110	Miss	110

Indeks	V	Tag	Dane
000	N		
001	N		
010	N		
011	N		
100	N		
101	N		
110	**Y**	**10**	**Mem[10110]**
111	N		

Przykład pamięci podręcznej

Adres słowa	Adres binarny	Hit/miss	Blok pamięci
26	11 010	Miss	010

Indeks	V	Tag	Dane
000	N		
001	N		
010	**Y**	**11**	**Mem[11010]**
011	N		
100	N		
101	N		
110	Y	10	Mem[10110]
111	N		

Przykład pamięci podręcznej

Adres słowa	Adres binarny	Hit/miss	Blok pamięci
22	10 110	Hit	110
26	11 010	Hit	010

Indeks	V	Tag	Dane
000	N		
001	N		
010	Y	11	Mem[11010]
011	N		
100	N		
101	N		
110	Y	10	Mem[10110]
111	N		

Przykład pamięci podręcznej

Adres słowa	Adres binarny	Hit/miss	Blok pamieci
16	10 000	Miss	000
3	00 011	Miss	011
16	10 000	Hit	000

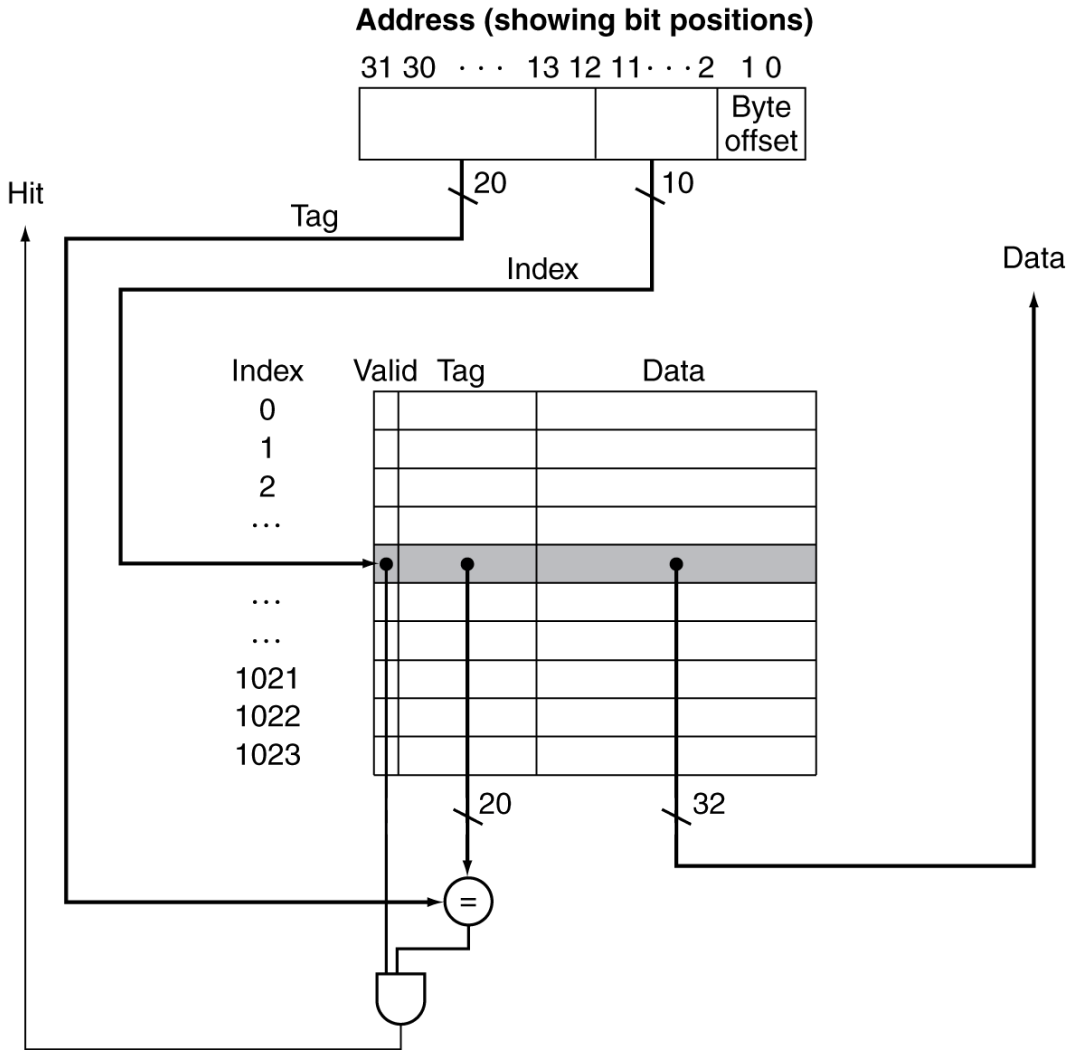
Indeks	V	Tag	Dane
000	**Y**	**10**	**Mem[10000]**
001	N		
010	Y	11	Mem[11010]
011	**Y**	**00**	**Mem[00011]**
100	N		
101	N		
110	Y	10	Mem[10110]
111	N		

Przykład pamięci podręcznej

Adres słowa	Adres binarny	Hit/miss	Blok pamięci
18	10 010	Miss	010

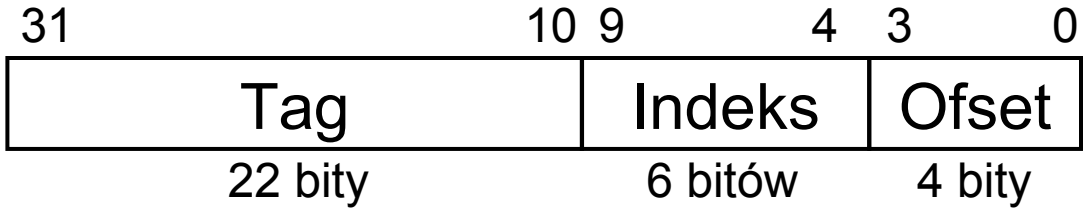
Indeks	V	Tag	Dane
000	Y	10	Mem[10000]
001	N		
010	**Y**	**10**	**Mem[10010]**
011	Y	00	Mem[00011]
100	N		
101	N		
110	Y	10	Mem[10110]
111	N		

Podział adresu



Przykład: większe bloki

- 64 bloki, 16 bajty/blok
  - Do którego numeru bloku odwzorowuje się adres 1200?
- Adres bloku = $\lfloor 1200/16 \rfloor = 75$
- Numer bloku = 75 modulo 64 = 11



Sprawa rozmiaru bloku

- Większe bloki powinny zmniejszać wskaźnik nietrafień (miss rate)
  - Ze względu na lokalność przestrzenną
- Ale pamięć podręczna jest ograniczona
  - Większe bloki ⇒ mniej bloków
    - Większa rywalizacja ⇒ zwiększony miss rate
  - Większe bloki ⇒ skażenie (pollution)
- Większy koszt nietrafienia
  - Może skompensować zmniejszenie miss rate
  - Pomaga wczesny restart i kolejność czytania

Nietrafienia pamięci podr.

- Jeśli trafienie, to CPU postępuje normalnie
- Jeśli nietrafienie pamięci podręcznej (cache miss)
  - Zatrzymaj potok CPU
  - Ładuj blok z następnego poziomu hierarchii
  - Nietrafienie pamięci podręcznej instrukcji
    - Restartuj ładowanie instrukcji
  - Nietrafienie pamięci podręcznej danych
    - Dokończ dostęp do danej

Write-Through (zapis poprzez)

- Przy trafieniu przy zapisie można tylko uaktualniać blok w pamięci podręcznej
  - Ale wtedy pamięć ta i pamięć główna są niezgodne
- Write through: uaktualnij także pamięć główną
- To jednak wydłuża operację zapisu
  - np. jeśli bazowe CPI = 1, 10% instrukcji to store, zapis do pamięci trwa 100 cykli
    - Efektywne CPI = 1 + 0.1×100 = 11
- Rozwiązanie: bufor zapisu (write buffer)
  - Przechowuje dane oczekujące zapisu do pamięci
  - CPU kontynuuje bez zatrzymania
    - Zatrzymanie tylko jeśli bufor zapisu jest zapełniony

Write-Back (zapis zwrotny)

- Alternatywa: Jeśli trafienie przy zapisie, to uaktualnij tylko blok w pamięci podręcznej
  - Pamiętaj „brudne” (dirty) bloki
- Gdy brudny blok jest wymieniany
  - Zapisz go zwrotnie do pamięci głównej (Write it back to memory)
  - Można użyć bufor zapisu by wpierw wczytać blok do wymienianej pozycji pamięci podręcznej

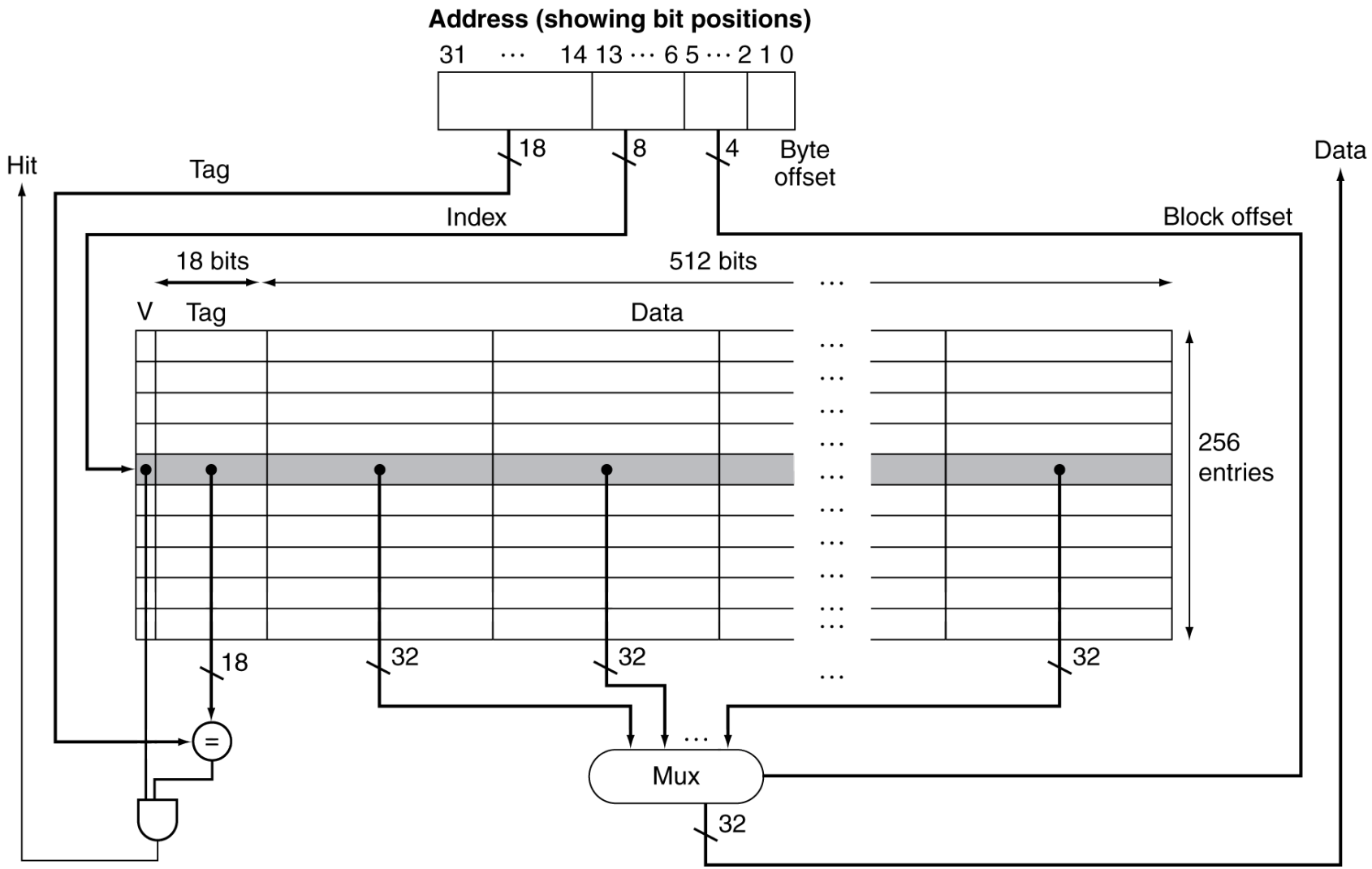
Alokacja przy zapisie

- Co może się dziać przy nietrafieniu przy zapisie?
- Rozwiązania dla zapisu poprzez
  - Alokacja przy nietrafieniu (Allocate on miss): ładuj blok
  - Zapisz naokoło (Write around): nie ładuj bloku
    - Gdyż programy często zapisują cały blok przed jego czytaniem (np. inicjalizacja)
- Dla zapisu zwrotnego (write-back)
  - Zwykle ładuj blok

Przykład: Intrinsity FastMATH

- Procesor MIPS – systemy wbudowane
  - 12-etapowy potok
  - W każdym cyklu dostęp do instrukcji i danych
- Podzielony cache: I-cache i D-cache
  - Każdy 16KB: 256 bloków × 16 słów/blok
  - D-cache: write-through lub write-back
- Wskaźniki nietrafień dla SPEC2000
  - I-cache: 0.4%
  - D-cache: 11.4%
  - Średnia ważona: 3.2%

Przykład: Intrinsity FastMATH



Pamięć podr. – pamięć główna

- Użyj DRAM'y do budowy pamięci głównej
  - Stała szerokość (np. 1 słowo)
  - Połączona taktowaną szyną (bus) o stałej szerokości
    - Zegar szyny jest zwykle wolniejszy od zegara CPU
- Przykład odczytu bloku do pamięci podręcznej
  - 1 cykl szyny na przesłanie adresu
  - 15 cykli szyny na dostęp do DRAM
  - 1 cykl szyny na transfer danych
- Dla 4-słowowego bloku i szerokiej na 1 słowo DRAM
  - Koszt nietrafienia = 1 + 4×15 + 4×1 = 65 cykli szyny
  - Pasmo = 16 bajtów / 65 cykli = 0.25 B/cykl

Wydajność pamięci podręcznej

- Składowe czasu CPU
  - Cykle wykonania programu
    - Włączając w to czasy trafienia pamięci podręcznej
  - Cykle zatrzymania ze względu na pamięć
    - Głównie z powodu nietrafień pamięci podręcznej
- Przy uproszczonych założeniach:

$$\text{Memory stall cycles} = \frac{\text{Memory accesses}}{\text{Program}} \times \text{Miss rate} \times \text{Miss penalty}$$

$$= \frac{\text{Instructions}}{\text{Program}} \times \frac{\text{Misses}}{\text{Instruction}} \times \text{Miss penalty}$$

Przykład

- Dane
  - Wskaźnik nietrafień I-cache = 2%
  - Wskaźnik nietrafień D-cache = 4%
  - Koszt nietrafienia = 100 cykli
  - Bazowy CPI (idealna pamięć podręczna) = 2
  - Load & stores stanowią 36% instrukcji
- Cykle nietrafienia na instrukcję
  - I-cache: 0.02 × 100 = 2
  - D-cache: 0.36 × 0.04 × 100 = 1.44
- Faktyczny CPI = 2 + 2 + 1.44 = 5.44
  - Idealne CPU jest 5.44/2 = 2.72 razy szybsze

Średni czas dostępu (AMAT)

- Czas trafienia jest także istotny dla wydajności
- Average memory access time (AMAT)
  - AMAT = Hit time + Miss rate × Miss penalty
- Przykład
  - CPU ma 1ns zegar, czas trafienia = 1 cykl, koszt nietrafienia = 20 cykli, wskaźnik nietrafień I-cache = 5%
  - AMAT = 1 + 0.05 × 20 = 2ns
    - 2 cykle na instrukcję

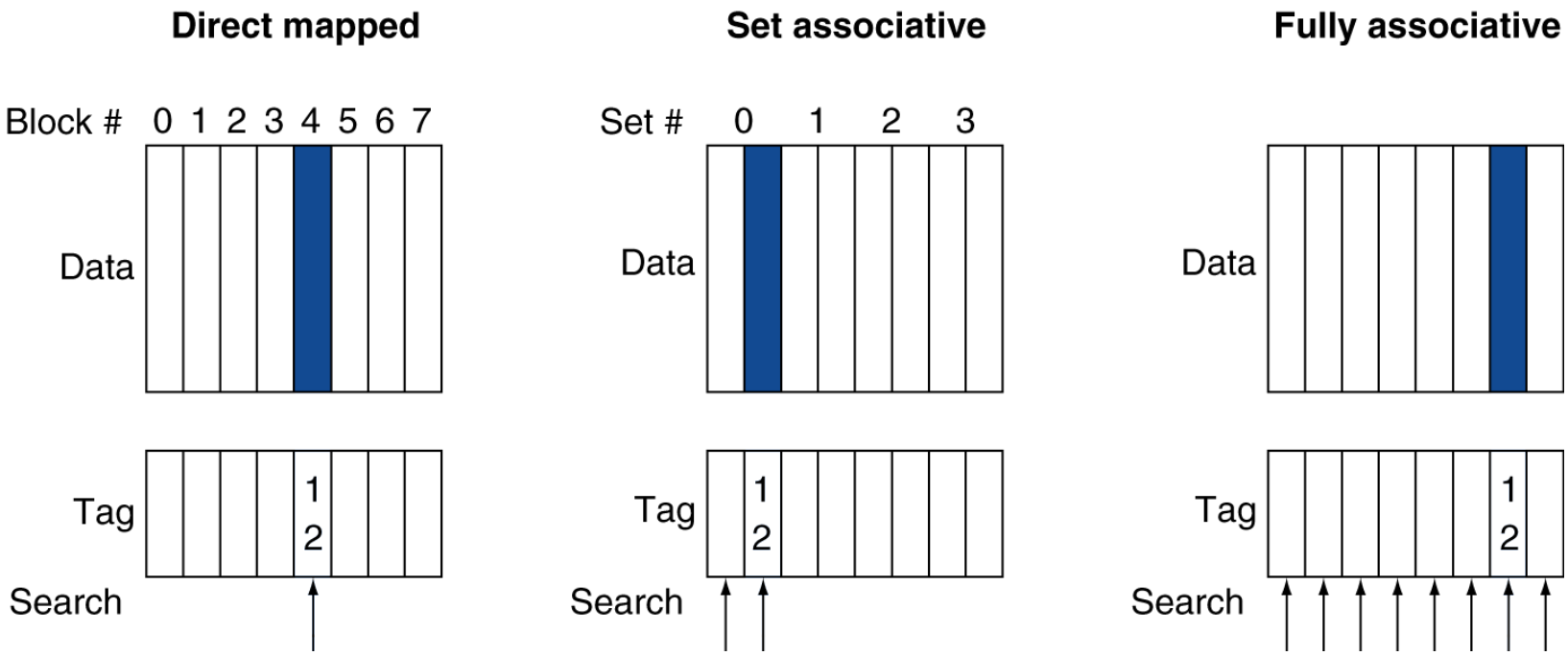
Wydajność – podsumowanie

- Gdy wzrasta wydajność CPU
  - Rośnie znaczenie kosztu nietrafienia
- Gdy zmniejszamy bazowe CPI
  - Zwiększa się udział czasu związany z zatrzymaniami powodowanymi przez pamięć
- Gdy zwiększamy szybkość zegara
  - Zatrzymania przez pamięć zajmują więcej cykli CPU
- Przy ocenie wydajności systemu należy uwzględniać zachowanie pamięci podr.

Asocjacyjne pamięci podr.

- W pełni asocjacyjne
  - Blok można lokować w dowolnym miejscu
  - Konieczne poszukiwanie go w całej pamięci
  - Komparator na każdą lokację (kosztowne)
- Asocjacyjne *n*-zbiory (*n*-way set associative)
  - Każdy zbiór liczy *n* elementów
  - Numer bloku określa który zbiór
    - (Numer bloku) modulo (#zbiorów w pamięci podr.)
  - Poszukuj go w całym wybranym zbiorze
  - *n* komparatorów (mniej kosztowne)

Przykład


Direct mapped
Block # 0 1 2 3 4 5 6 7
Data
Tag
1
2
Search
Set associative
Set # 0 1 2 3
Data
Tag
1
2
Search
Fully associative
Data
Tag
1
2
Search

Zakres asocjacyjności

- Dla pamięci podręcznej z 8 lokacjami

One-way set associative (direct mapped)

Block	Tag	Data
0		
1		
2		
3		
4		
5		
6		
7		

Two-way set associative

Set	Tag	Data	Tag	Data
0				
1				
2				
3				

Four-way set associative

Set	Tag	Data	Tag	Data	Tag	Data	Tag	Data
0								
1								

Eight-way set associative (fully associative)

Tag	Data	Tag	Data	Tag	Data	Tag	Data	Tag	Data	Tag	Data	Tag	Data	Tag	Data

Przykład

- Porównamy 4 blokowe pamięci podręczne
  - Odwzorowaną bezpośrednio, asocjacyjną z 2-zbiorami i w pełni asocjacyjną
  - Sekwencja dostępów do bloków: 0, 8, 0, 6, 8
- Odwzorowana bezpośrednio

Adres bloku	Indeks	Hit/miss	Zawartość pamięci podręczne po dostępie			
			0	1	2	3
0	0	miss	**Mem[0]**			
8	0	miss	**Mem[8]**			
0	0	miss	**Mem[0]**			
6	2	miss	Mem[0]		**Mem[6]**	
8	0	miss	**Mem[8]**		Mem[6]	

Przykład

- Asocjacyjna z 2-zbiorami

Adres bloku	Indeks	Hit/miss	Zawartość pamięci podręczne po dostępie			
			Zbiór 0		Zbiór 1	
0	0	miss	**Mem[0]**			
8	0	miss	Mem[0]	**Mem[8]**		
0	0	hit	**Mem[0]**	Mem[8]		
6	0	miss	Mem[0]	**Mem[6]**		
8	0	miss	**Mem[8]**	Mem[6]		

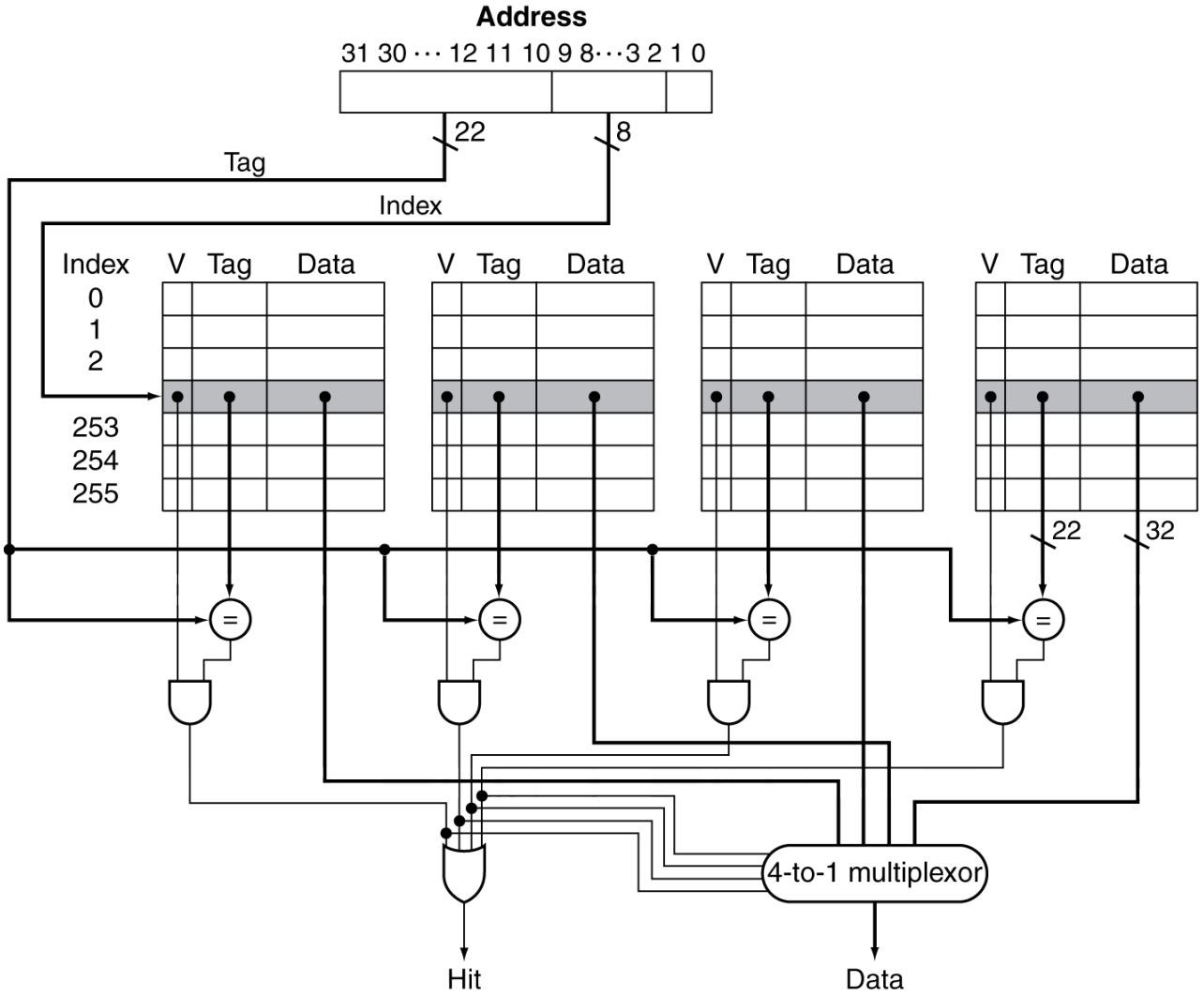
- W pełni asocjacyjna

Adres bloku		Hit/miss	Zawartość pamięci podręczne po dostępie			
0		miss	**Mem[0]**			
8		miss	Mem[0]	**Mem[8]**		
0		hit	**Mem[0]**	Mem[8]		
6		miss	Mem[0]	Mem[8]	**Mem[6]**	
8		hit	Mem[0]	**Mem[8]**	Mem[6]	

Jak wiele asocjacyjności

- Zwiększona asocjacyjność zmniejsza wskaźnik nietrafień
  - Ale ze zmniejszającym się zyskiem
- Symulacja systemu z 64KB D-cache, 16 słów na blok, SPEC2000
  - 1-zbiory: 10.3%
  - 2-zbiory: 8.6%
  - 4-zbiory: 8.3%
  - 8-zbiory: 8.1%

Organizacja pamięci z *n*-zbiorami



Polityka wymiany bloków

- Odwzorowanie bezpośrednie: oczywista
- Zbiory asocjacyjne
  - Wybierz pustą lokację, o ile jest
  - Jeśli nie ma, wybierz lokację ze zbioru
- Least-recently used (LRU)
  - Wybierz lokację najdłużej nieużywaną
    - Proste dla 2-zbiorów, tolerowalne dla 4-zbiorów, zbyt złożone dla większych zbiorów
- Wybór przypadkowy (Random)
  - Dla dużych zbiorów daje prawie taką samą efektywność jak LRU

Wielopoziomowe pam. podr.

- Główna pamięć podręczna jest przy CPU
  - Mała, ale szybka
- Pamięć drugiego poziomu (Level-2 cache) obsługuje nietrafienia pamięci głównej
  - Większa, wolniejsza, ale ciągle szybsza od pamięci głównej
- Pamięć główna obsługuje nietrafienia pamięci podręcznej L-2
- Niekiedy jest też pamięć podręczna L-3

Przykład

- Dane
  - CPU ma bazowe CPI = 1, zegar = 4GHz
  - Wskaźnik nietrafień dla instrukcji = 2%
  - Czas dostępu do pamięci głównej = 100ns
- Tylko główna pamięć podręczna
  - Koszt nietrafienia = 100ns/0.25ns = 400 cykli
  - Efektywne CPI = 1 + 0.02 × 400 = 9

Przykład, c.d.

- Dodajemy pamięć L-2
  - Czas dostępu = 5ns
  - Wskaźnik nietrafień pamięci głównej = 0.5%
- Nietrafienie L-1, trafienie L-2
  - Koszt = 5ns/0.25ns = 20 cykli
- Nietrafienie obu pamięci podręcznych
  - Dodatkowy koszt = 400 cykli
- CPI = 1 + 0.02 × 20 + 0.005 × 400 = 3.4
- Stosunek wydajności = 9/3.4 = 2.6

Pożądane właściwości L-1 i L-2

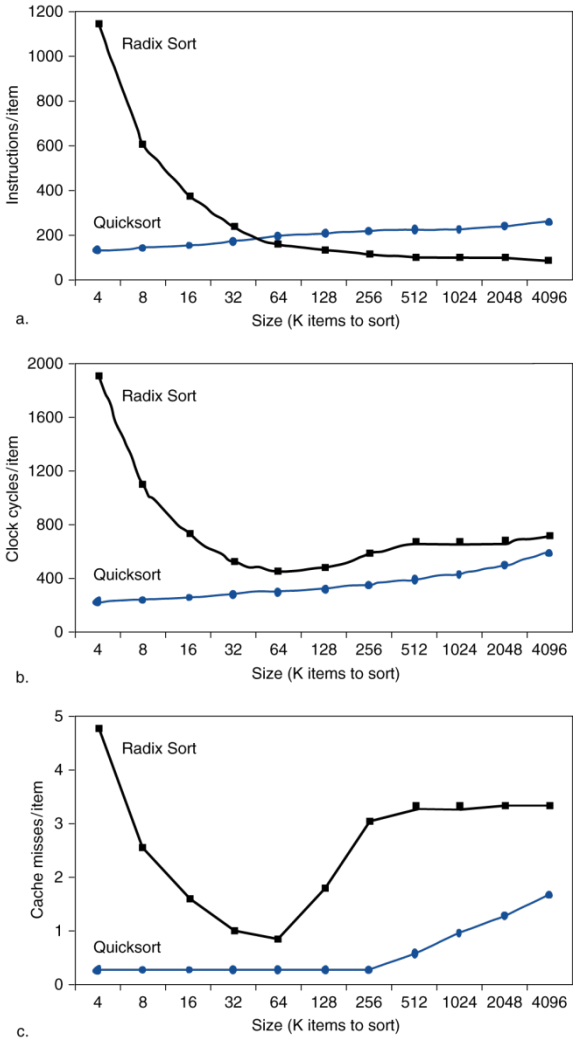
- Główna pamięć podręczna
  - Przede wszystkim minimalny czas trafienia
- Pamięć L-2
  - Przede wszystkim niski wskaźnik nietrafień – minimum dostępów do pamięci głównej
  - Czas trafienia mniej istotny
- Wynik
  - L-1 zwykle mniejsza niż w systemie bez L-2
  - Rozmiar bloku L-1 mniejszy niż bloku L-2

Interakcje z zaawansowanymi CPU

- Out-of-order CPU może wykonywać instrukcje gdy nietrafienie pamięci podr.
  - Wykonywany store pozostaje w load/store unit
  - Zależne instrukcje czekają w reservation stations
    - Instrukcje niezależne wykonują się
- Efekt nietrafienia zależy od przepływu danych programu
  - O wiele trudniejsze do analizy
  - Korzystaj z symulacji systemu

Interakcje z software'em

- Nietrafienia zależą od wzorców dostępu do pamięci
  - Charakter algorytmu
  - Optymalizacji kompilatora ze względu na dostęp do pamięci



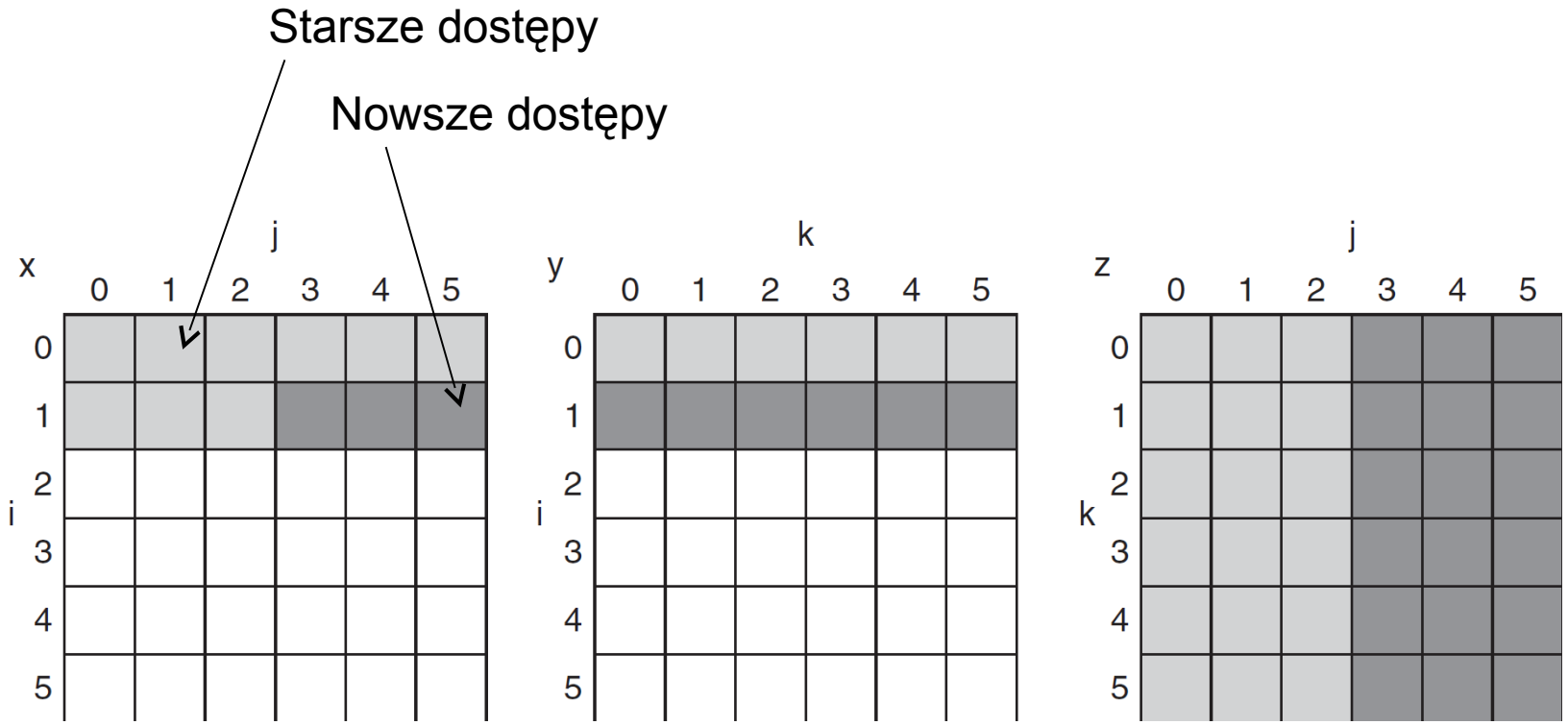
Optymalizacja przez bloki transferów

- Cel: maksymalizacja dostępu do danych zanim zostaną wymienione
- Rozważ wewnętrzne pętle DGEMM:

```
for (int j = 0; j < n; ++j)
{
  double cij = C[i+j*n];
  for( int k = 0; k < n; k++ )
    cij += A[i+k*n] * B[k+j*n];
  C[i+j*n] = cij;
}
```

Wzór dostępów DGEMM

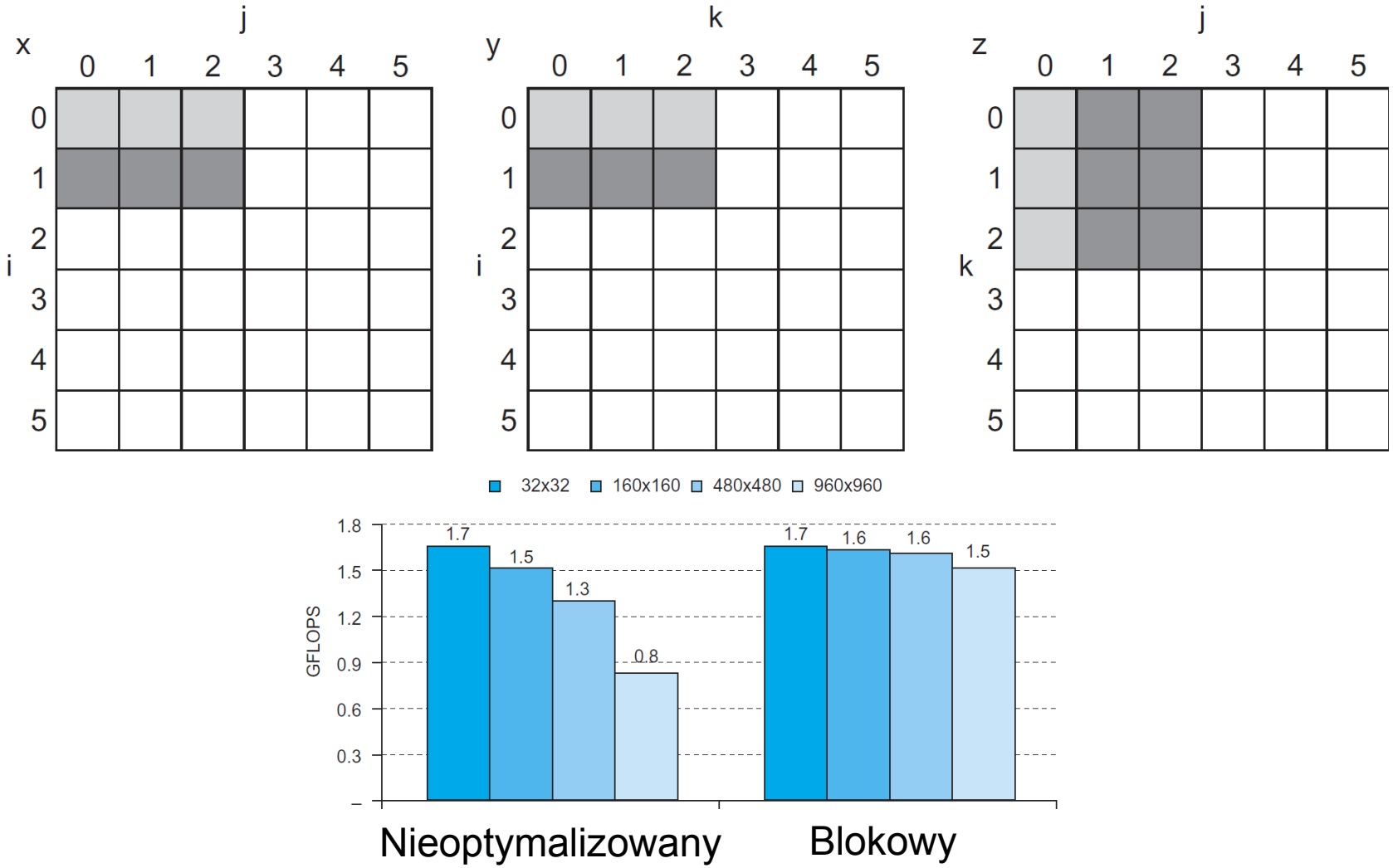
- Macierze C, A i B



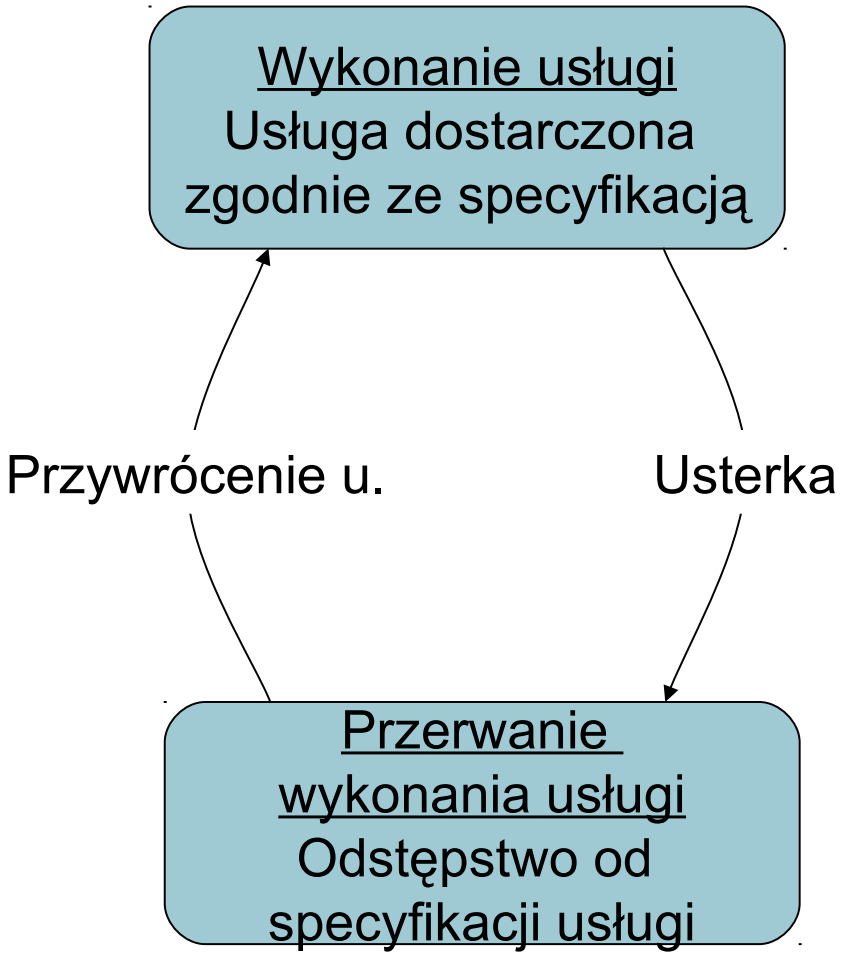
Blokowy DGEMM

```
1 #define BLOCKSIZE 32
2 void do_block (int n, int si, int sj, int sk, double *A, double
3 *B, double *C)
4 {
5  for (int i = si; i < si+BLOCKSIZE; ++i)
6   for (int j = sj; j < sj+BLOCKSIZE; ++j)
7   {
8    double cij = C[i+j*n];/* cij = C[i][j] */
9    for( int k = sk; k < sk+BLOCKSIZE; k++ )
10    cij += A[i+k*n] * B[k+j*n];/* cij+=A[i][k]*B[k][j] */
11   C[i+j*n] = cij;/* C[i][j] = cij */
12  }
13 }
14 void dgemm (int n, double* A, double* B, double* C)
15 {
16  for ( int sj = 0; sj < n; sj += BLOCKSIZE )
17   for ( int si = 0; si < n; si += BLOCKSIZE )
18    for ( int sk = 0; sk < n; sk += BLOCKSIZE )
19     do_block(n, si, sj, sk, A, B, C);
20 }
```

Dostępy blokowego DGEMM



Rzetelność (Dependability)



- Usterka: defekt elementu
  - Może ale nie musi prowadzić do awarii systemu

Miary rzetelności

- Niezawodność: mean time to failure (MTTF)
- Przerwa naprawcza: mean time to repair (MTTR)
- Średni czas między awariami
  - MTBF = MTTF + MTTR
- Dostępność = MTTF / (MTTF + MTTR)
- Poprawianie dostępności
  - Zwiększanie MTTF: unikanie usterek, odporność na usterki, przewidywanie usterek
  - Zmniejszanie MTTR: udoskonalanie narzędzi i procesów diagnostycznych i naprawczych

Kod Hamminga (SEC)

- Odległość Hamminga
  - Liczba pozycji na których różnią się dwie sekwencje (zwykle bitowe)
- Odległość minimalna = 2 pozwala na wykrycie pojedynczego błędu bitowego
  - Np. sekwencja z bitem parzystości
- Odległość minimalna = 3 pozwala na korekcję pojedynczego błędu, albo detekcję dwóch
- SEC – single error correcting

Kodowanie SEC

- W celu wyliczenia kodu Hamminga:
  - Ponumeruj bity od 1 z lewej
  - Wszystkie pozycje będące potęgą 2 zawierają bity parzystości
  - Każdy bit parzystości sprawdza część bitów:

Bit position		1	2	3	4	5	6	7	8	9	10	11	12
Encoded date bits		p1	p2	d1	p4	d2	d3	d4	p8	d5	d6	d7	d8
Parity bit coverate	p1	X		X		X		X		X		X	
	p2		X	X			X	X			X	X	
	p4				X	X	X	X					X
	p8								X	X	X	X	X

Dekodowanie SEC

- Wartości bitów parzystości wskazują które bity są błędne
  - Użyj numerowania z procedury kodowania
  - Np.
    - Bity parzystości = 0000 oznaczają brak błędu
    - Bity parzystości = 1010 wskazują, że bit 10 został przekłamany

Kod SEC/DEC

- Dodaj bit parzystości dla całego słowa (p_n)
- Uczyń odległość Hamminga = 4
- Dekodowanie:
  - Niech H = bity parzystości SEC
    - H parzyste, p_n parzyste: brak błędu (parzyste, czyli 0)
    - H nieparzyste, p_n nieparzyste: korygowalny pojedynczy błąd
    - H parzyste, p_n nieparzyste: błąd w bicie p_n
    - H nieparzyste, p_n parzyste, wystąpił błąd podwójny
- Uwaga: ECC DRAM używa SEC/DEC z 8 bitami dla ochrony każdych 64 bitów

Maszyny wirtualne (virtual m.)

- Komputer (gospodarz – host) emuluje zasoby i system operacyjny maszyny „gościa”
  - Udoskonalona izolacja wielu gości
  - Unika się problemów z niezawodnością i bezpieczeństwem
  - Wspomaga dzielenie zasobów
- Wirtualizacja ma pewien wpływ na wydajność
  - Wykonalne z nowoczesnymi wydajnymi komputerami
- Przykłady
  - IBM VM/370 (technologia z lat 1970!)
  - VMWare
  - Microsoft Virtual PC

Monitor maszyny wirtualnej

- Odwzorowuje zasoby wirtualne na rzeczywiste
  - Pamięć, urządzenia we/wy (I/O), procesory
- Kod gościa wykonuje się w trybie użytkownika (user mode)
  - Wychodzi do VMM (virtual machine monitor) przy instrukcjach administratora i próbie dostępu do zasobów chronionych
- OS gościa może być inny niż gospodarza
- VMM działa na rzeczywistych urz. we/wy
  - Emuluje wirtualne urządzenia we/wy gości

Przykł.: wirtualizacja licznika

- Normalnie przy przerwaniu od licznika
  - OS zawiesza proces bieżący, obsługuje przerwanie i aktywuje następny proces
- Przy monitorze maszyny wirtualnej
  - VMM zawiesza bieżącą VM, obsługuje przerwanie, wybiera i aktywuje następną VM
- Jeśli VM wymaga przerwań od licznika
  - VMM emuluje licznik wirtualny
  - Emuluje przerwanie dla VM (virtual machine) jeśli wystąpi fizyczne przerwanie od licznika

Wsparcie dla listy instrukcji

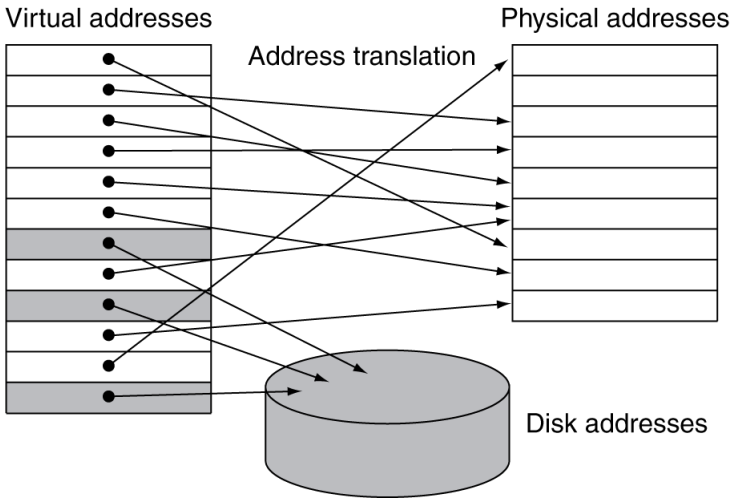
- Tryby użytkownika i systemowy (admin.)
- Instrukcje administratora (privileged) dostępne tylko w trybie systemowym
  - Wyjście (trap) do systemu jeśli wykonywane w trybie użytkownika
- Wszystkie zasoby fizyczne dostępne tylko przez instrukcje systemowe
  - W tym tablice stron, sterowanie przerwaniami, rejestry we/wy
- Renesans wsparcia dla wirtualizacji
  - Współczesne ISA (np. x86) adaptują się

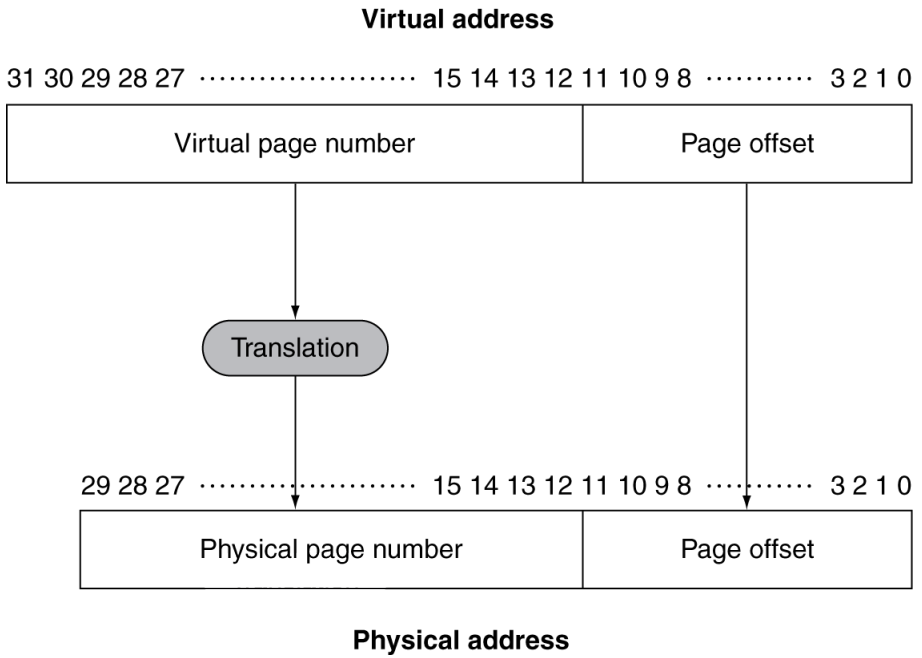
Pamięć wirtualna

- Użyj pamięć główną jak pamięć podręczną dla pamięci wtórnej (dyskowej)
  - Dokonywane wspólnie przez sprzęt CPU i system operacyjny (OS)
- Programy dzielą się pamięcią główną
  - Każdy dostaje prywatną wirtualną przestrzeń adresową dla często używanych danych i kodu
  - Chronioną przed innymi programami
- CPU i OS tłumaczy (translates) adres wirtualny na fizyczny (VM – virtual memory)
  - „Blok” VM nazywany jest stroną (page)
  - Nietrafienie przy translacji VM: błąd strony

Translacja adresu

- Ustalone rozmiary stron (np. 4K)





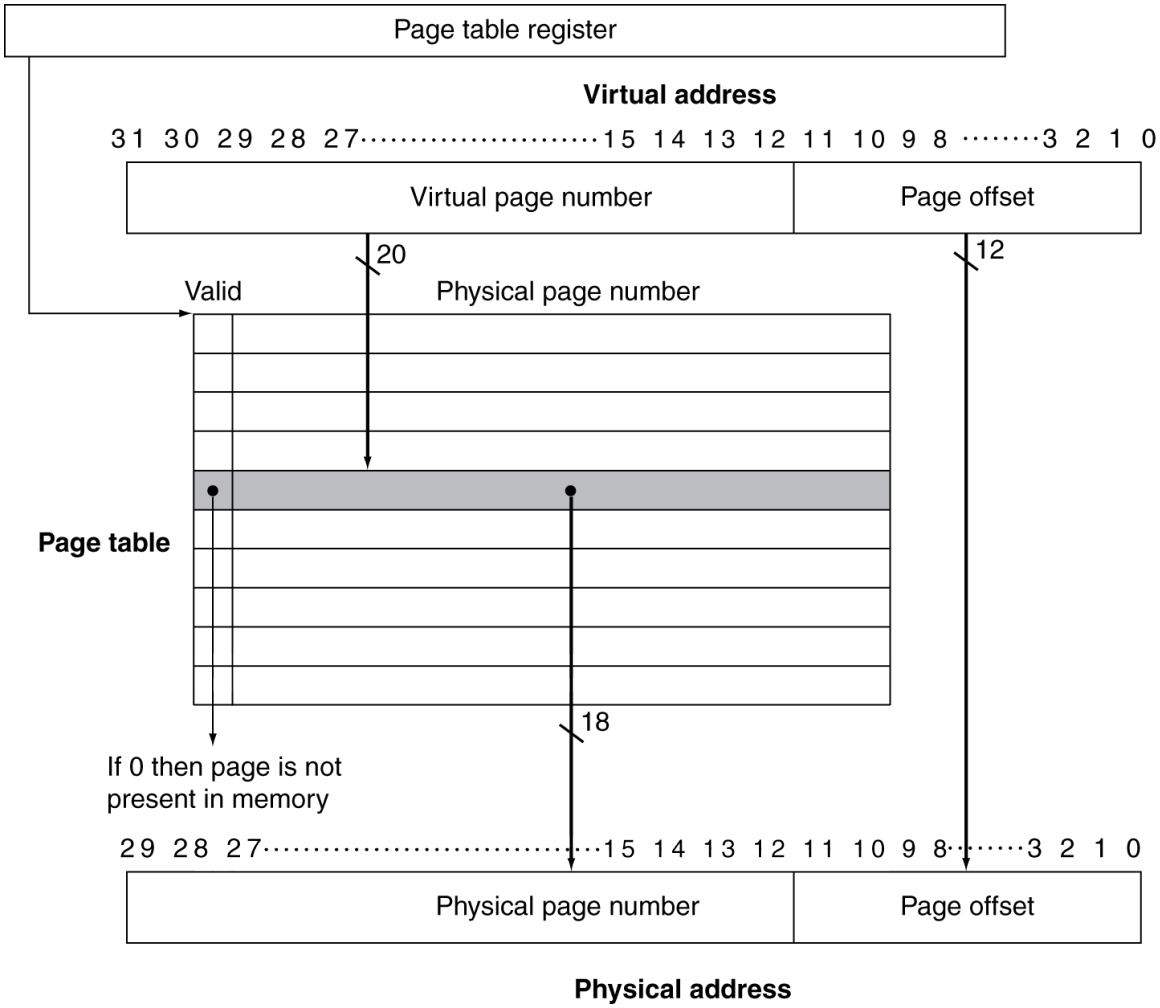
Koszt błędu strony

- Przy błędzie strony (page fault) stronę trzeba pobrać z dysku
  - Trwa to miliony cykli zegarowych
  - Obsługiwane przez kod OS
- Staraj się minimalizować wskaźnik błędów strony (page fault rate)
  - W pełni asocjacyjna lokalizacja (placement) stron
  - „Inteligentne” algorytmy wymiany stron

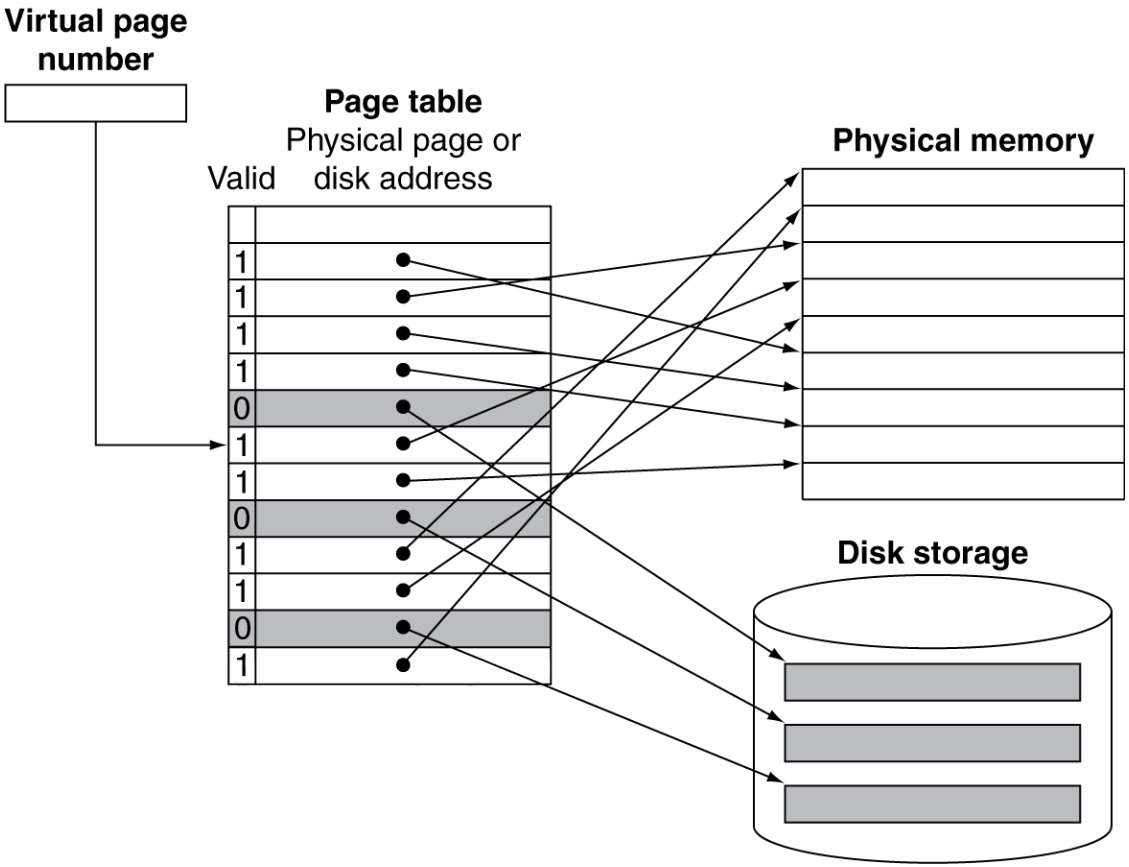
Tablice stron (page tables)

- Przechowują informacje o lokalizacji
  - Ich elementy (PTE) zawierają dane o stronach indeksowane numerem strony wirtualnej
  - Rejestr tablicy stron w CPU wskazuje tablicę stron w pamięci fizycznej
- Jeśli strona znajduje się w pamięci
  - PTE przechowuje numer fizyczny strony
  - I inne bity stanu (referenced, dirty, …)
- Jeśli nie
  - PTE może wskazywać jej miejsce na dysku w przestrzeni „swap” (E w PTE: entry)

Translacja przez tablicę stron



Strony a pamięć



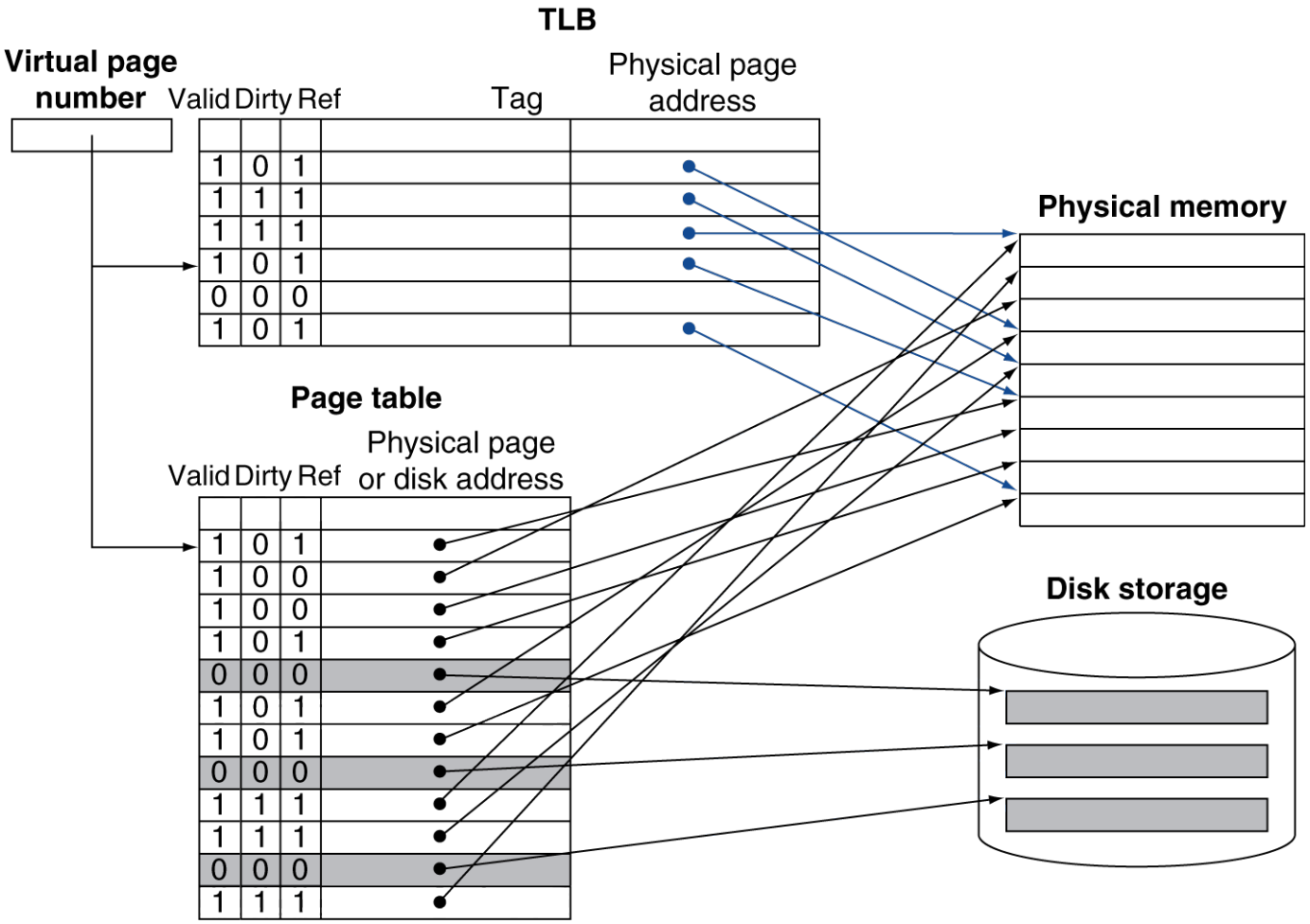
Wymiana i zapis

- W celu zmniejszenia wskaźnika błędów stron wymiana *least-recently used* (LRU)
  - Bit odsyłacza (reference, też use bit) w PTE ustaw na 1 gdy dostęp do strony
  - Okresowo zerowany przez OS
  - Strona z bitem odsyłacza = 0 nie była ostatnio używana
- Zapis dysku trwa miliony cykli
  - Cały blok, nie indywidualne lokacje
  - Zapis poprzez nie jest sensowny
  - Używaj zapisu zwrotnego
  - Ustaw bit „dirty” w PTE przy zapisie strony

Szybka translacja dzięki TLB

- Translacja adresu wydaje się wymagać więcej adresowań pamięci
  - Jedno by dotrzeć do PTE
  - Po czym właściwy dostęp do pamięci
- Ale dostępy do tablicy stron są silnie lokalne
  - Tak więc użyj szybkiej pamięci podręcznej dla PTE wewnątrz CPU
  - Nazywanej *Translation Look-aside Buffer* (TLB)
  - Zwykle: 16–512 PTE, 0.5–1 cykli dla trafienia, 10–100 cykli dla nietrafienia, wskaźnik nietrafień 0.01%–1%
  - Nietrafienia mogą być obsługiwane przez sprzęt lub oprogramowanie

Szybka translacja dzięki TLB



Nietrafienia TLB

- Jeśli strona jest w pamięci
  - Załaduj PTE z pamięci i spróbuj znowu
  - Może być obsługiwane przez sprzęt
    - Może być skomplikowane przy bardziej złożonej strukturze tablicy stron
  - Lub przez oprogramowanie
    - Specjalny wyjątek z optymalizowaną obsługą
- Jeśli nie (błąd strony)
  - OS dokonuje ładowania strony i uaktualnia tablicę stron
  - Po czym restartuje problematyczną instrukcję

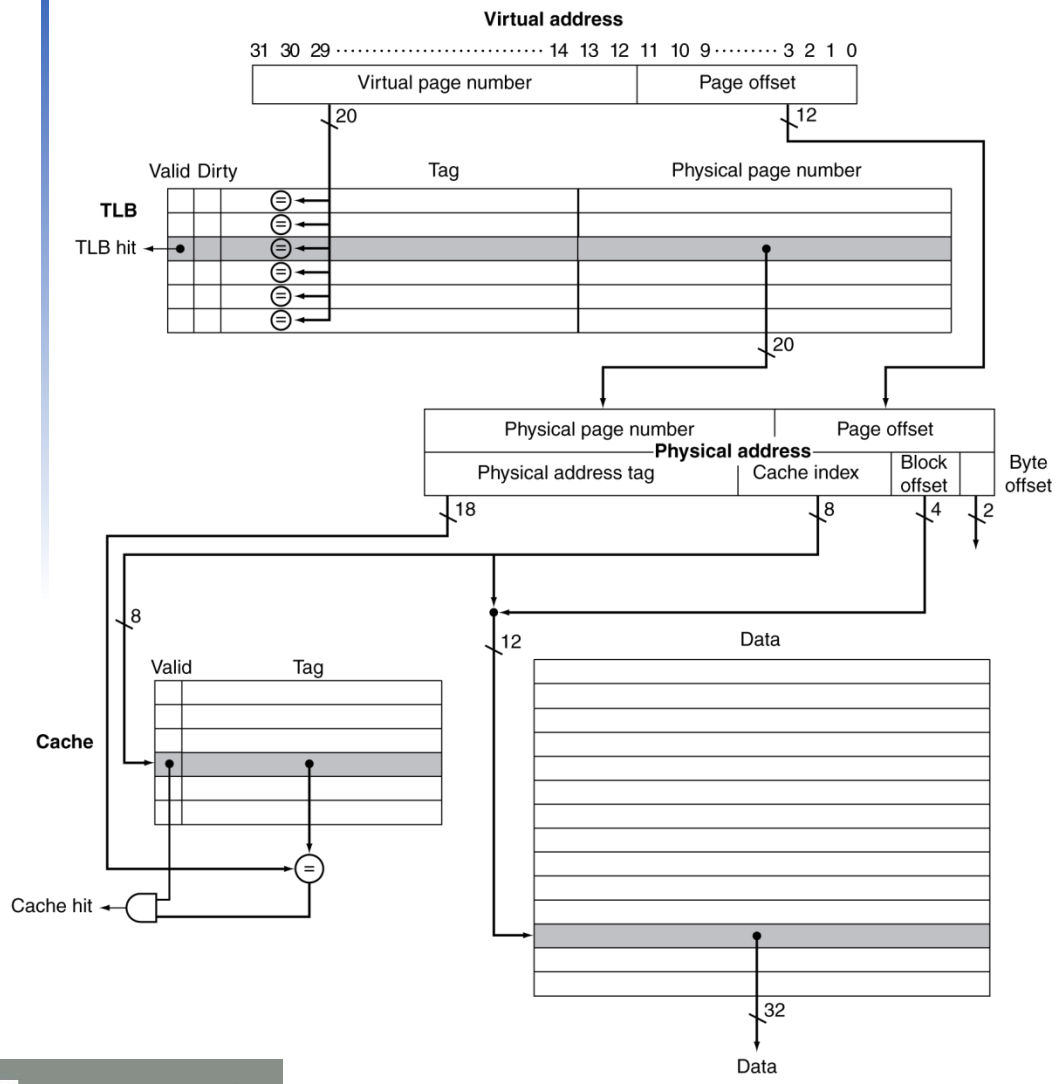
Obsługa nietrafień TLB

- Nietrafienie TLB oznacza
  - Strona dostępna, ale brak PTE w TLB
  - Strona niedostępna
- Musi rozpoznawać nietrafienie TLB zanim rejestr przeznaczenia zostanie nadpisany
  - Spowoduj wyjątek
- Obsługa kopiuje PTE z pamięci do TLB
  - Po czym restartuje instrukcję
  - Jeśli strona niedostępna następuje błąd strony

Obsługa błędu strony

- Użyj problematycznego adresu wirtualnego do znalezienia PTE
- Zlokalizuj stronę na dysku
- Wybierz stronę do podmiany
  - Jeśli „dirty”, wpierw zapisz ją na dysku
- Wczytaj stronę do pamięci i uaktualnij tablicę stron
- Przywróć procesowi wykonywalność
  - Restartuj od problematycznej instrukcji

Interakcja TLB – pamięć podr.



- Jeśli tag p. p. używa adresu fizycznego
  - Potrzebna translacja przed zajrzeniem do pamięci podręcznej
- Alternatywa: użyj adresu wirtualnego jako tag
  - Komplikacje przez nakładanie się
    - Różne adresy wirtualne dla współdzielonej pamięci fizycznej

Ochrona pamięci

- Zadania mogą współużytkować (share) części ich wirtualnej przestrzeni adresowej
  - Konieczna ochrona przed błędnym dostępem
  - Wymaga pomocy ze strony OS
- Wsparcie sprzętowe dla ochrony przez OS
  - Privileged supervisor mode (też kernel mode)
  - Privileged instructions (instr. administratora)
  - Tablice stron i inne info o stanie dostępne tylko w trybie supervisor (nadrzędnym)
  - Wyjątki wywołujące system (MIPS: syscall)

Hierarchia pamięci

The BIG Picture

- Na wszystkich poziomach hierarchii obowiązują te same zasady
  - Oparte na idei pamięci podręcznej
- Na każdym poziomie hierarchii mamy
  - Alokacja bloku (block placement)
  - Znajdowanie bloku
  - Wymianę przy nietrafieniu (replacement)
  - Politykę zapisu (write policy)

Alokacja bloku

- Wynika ze stopnia asocjacyjności
  - Odwzorowanie bezpośrednie (1-way associative)
    - Tylko jedna możliwość alokacji
  - Asocjacyjne n-zbiory (n-way set associative)
    - n możliwości alokacji wewnątrz zbioru
  - Pełna asocjacyjność
    - Dowolna alokacja
- Asocjacyjność zmniejsza miss rate
  - Zwiększa złożoność, koszt i czas dostępu

Znajdowanie bloku

Asocjacyjność	Metoda alokacji	Porównania tagu
Odwzorowanie b.	Indeks	1
Asocjacyjne n-zbiory	Indeks zbioru, po czym szukanie w zbiorze	n
Pełna asocjacyjność	Przeglądaj całość	#elementów
	Pełna lookup table	0

- Sprzętowe pamięci podręczne
  - Zmniejszaj liczbę porównań w celu redukcji kosztu
- Pamięć wirtualna
  - Pełna lookup table (tablica przeglądowa) czyni pełną asocjacyjność możliwą
  - Efekt w postaci zredukowanego wskaźnika nietrafień

Wymiana bloku

- Wybór bloku do wymiany przy nietrafieniu
  - Najmniej ostatnio używany (Least recently used, LRU)
    - Złożony i kosztowny sprzęt przy wysokiej asocjacyjności
  - Przypadkowy
    - Efekt zbliżony do LRU, łatwiejszy w implementacji
- Pamięć wirtualna
  - Aproksymacja LRU ze wsparciem sprzętowym

Polityka zapisu

- Zapis poprzez (write-through)
  - Uaktualnianie obu poziomów pamięci
  - Upraszcza wymianę bloku, ale może wymagać bufora zapisu (write buffer)
- Zapis zwrotny (write-back)
  - Uaktualnianie tylko górnego poziomu pamięci
  - Dolny poziom uaktualniany przy wymianie bloku
  - Potrzebna większa wiedza o stanie
- Pamięć wirtualna
  - Zważywszy na opóźnienie zapisu dysku tylko zapis zwrotny ma sens

Źródła nietrafień

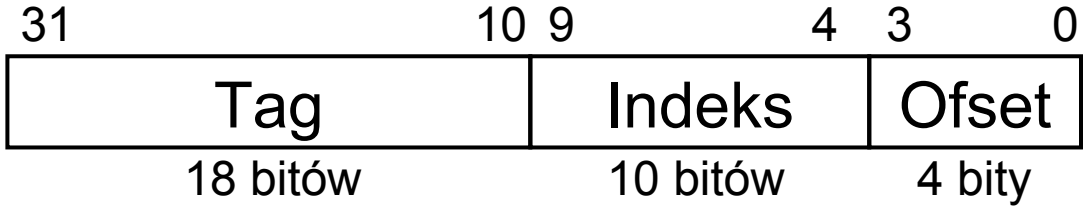
- Nietrafienia przymusowe (compulsory misses, też cold start misses)
  - Pierwszy dostęp do bloku
- Ograniczona pojemność (capacity misses)
  - Z powodu ograniczonego rozmiaru p. p.
  - Usunięty blok bywa ponownie potrzebny
- Konflikt (conflict misses, collision misses)
  - W nie w pełni asocjacyjnych pamięciach podr.
  - Na skutek konkurencji o miejsce w n-zbiorze
  - Nie wystąpi w pamięci podręcznej w pełni asocjacyjnej o tej samej wielkości

Kompromisy projektowe

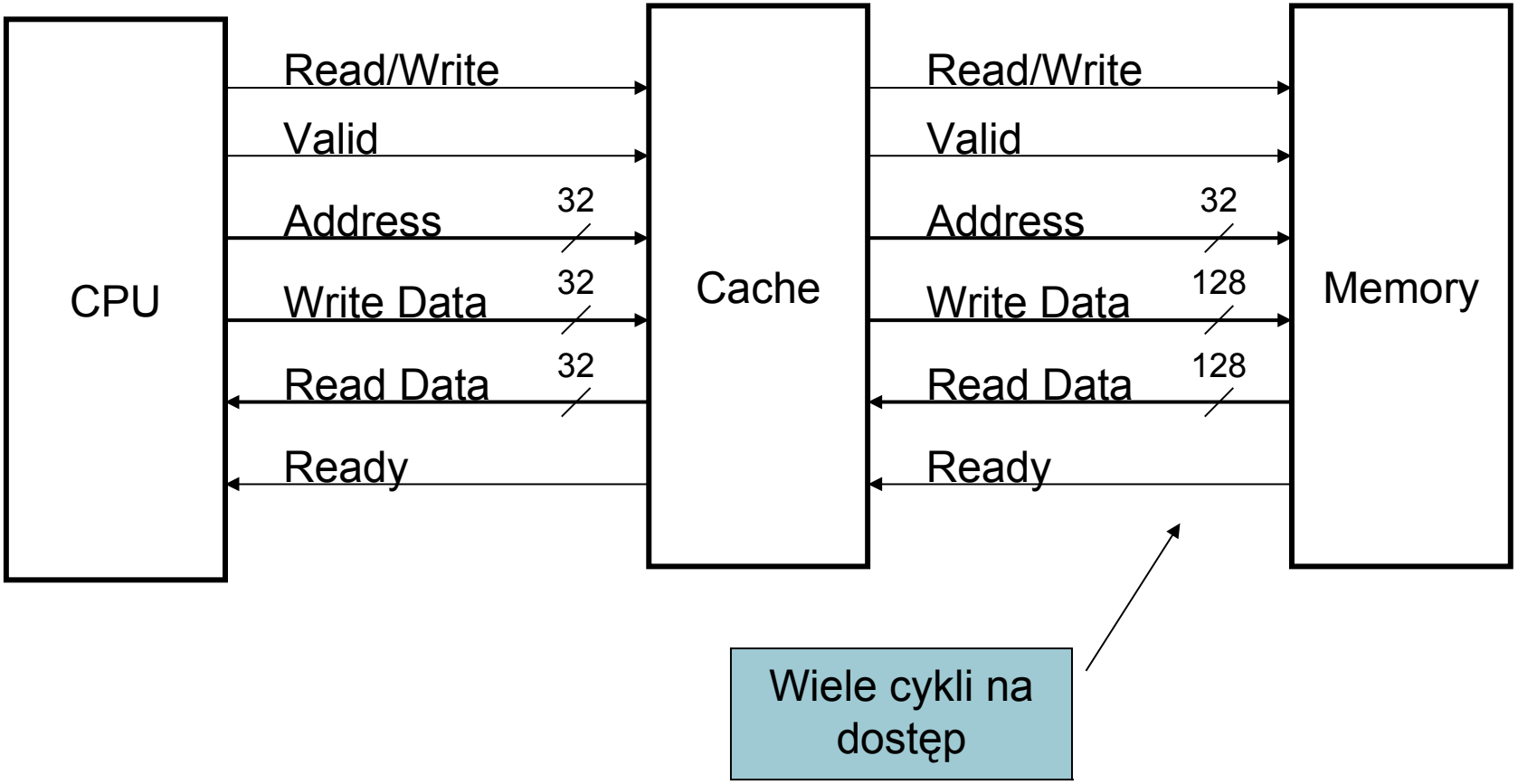
Zmiana projektowa	Wpływa na wskaźnik nietrafień	Ujemny wpływ na wydajność
Zwiększ pojemność pamięci podręcznej	Zmniejsza nietrafienia przez ogr. pojemność	Może zwiększyć czas dostępu
Zwiększ asocjacyjność	Zmniejsza nietrafienia przez konflikt	Może zwiększyć czas dostępu
Zwiększ rozmiar bloku	Zmniejsza nietrafienia przymusowe	Zwiększa koszt nietrafienia. Dla bardzo dużych rozmiarów bloku może zwiększać wskaźnik nietrafień przez skażenie.

Sterowanie pamięcią podr.

- Przykładowa charakterystyka pamięci podr.
  - Odwzorowanie bezpośrednie, zapis zwrotny, alokacja przy zapisie
  - Wielkość bloku: 4 słowa (16 bajtów)
  - Rozmiar pamięci: 16 KB (1024 bloki)
  - Adresy 32-bitowe
  - Bity valid i dirty dla każdego bloku
  - Pamięć blokująca (blocking cache)
    - CPU czeka aż do zakończenia dostępu

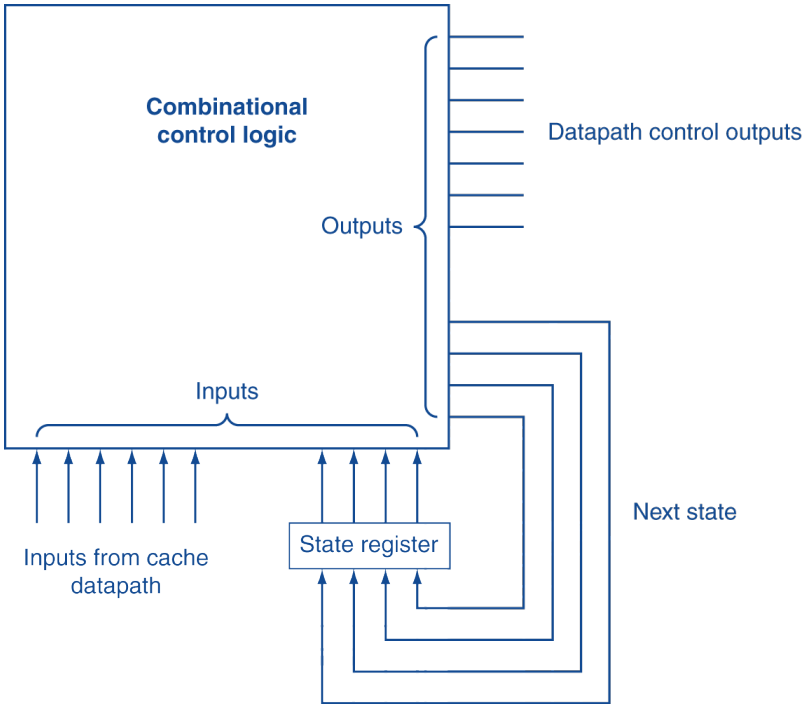


Sygnały interfejsu

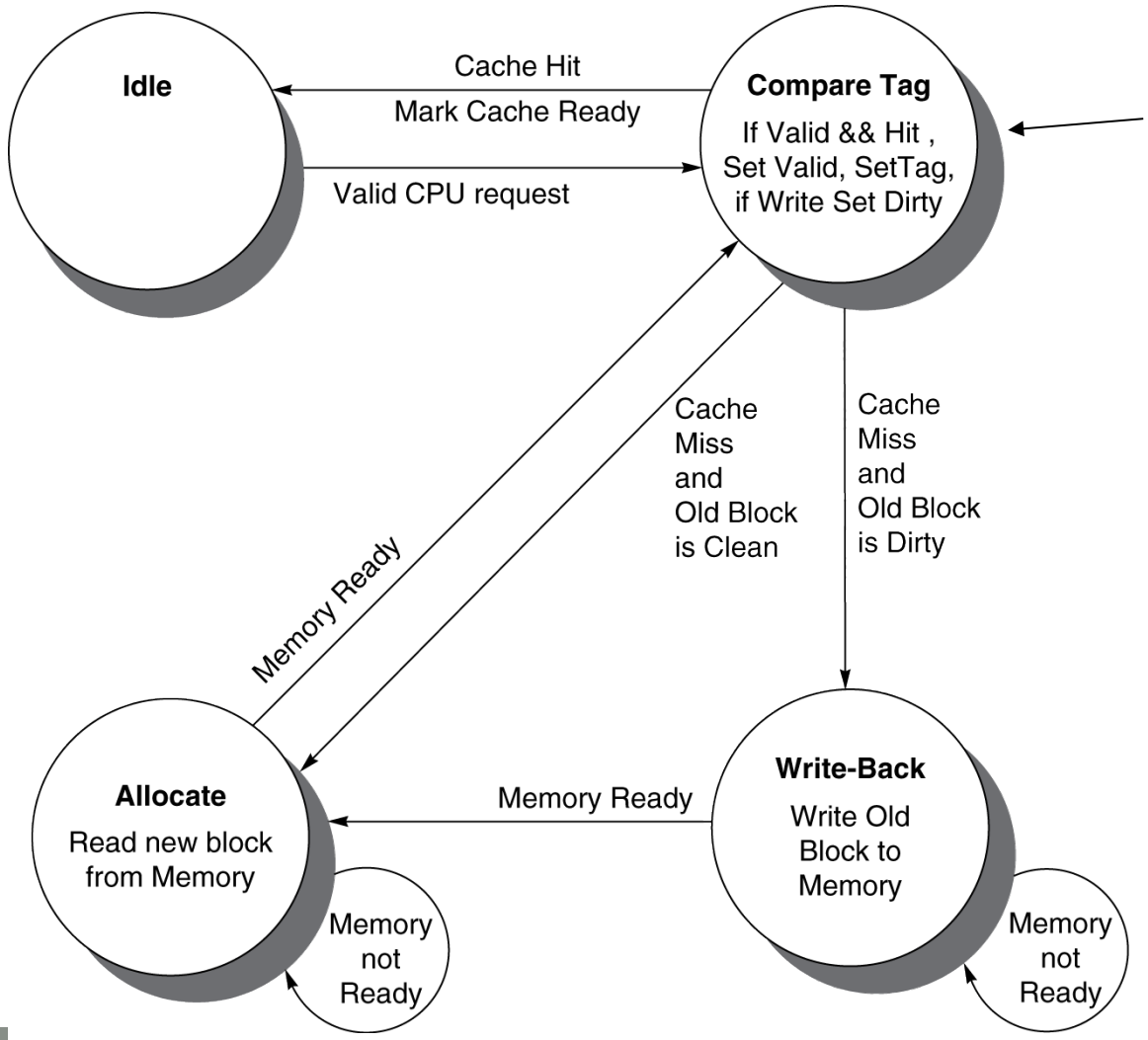


Automaty skończone

- Realizacja kroków sterowania automatem
- Zbiór stanów, zmiana na zboczu zegara
  - Stany są kodowane binarnie
  - Kod bieżącego stanu jest w rejestrze
  - Następny stan = f_n (bieżący stan, bieżące wejścia)
- Sterujące sygnały wyj. = f_o (bieżący stan)



Automat sterownika pam. p.



Można podzielić na osobne stany w celu redukcji cyklu zegara

Problem koherencji p. p.

- Załóżmy, że dwa rdzenie CPU dzielą wspólną pamięć fizyczną
  - Pamięci podręczne z zapisem poprzez

Krok	Zdarzenie	p. p. CPU A	p. p. CPU B	Pamięć
0				0
1	CPU A czyta X	0		0
2	CPU B czyta X	0	0	0
3	CPU A pisze 1 w X	1	0	1

Definicja koherencji

- Nieformalnie: Odczyty dają ostatnio zapisaną wartość
- Formalnie:
  - P zapisuje X; P czyta X (bez akcji pomiędzy)
    ⇒ odczyt zwraca zapisaną wartość
  - P_1 zapisuje X; P_2 czyta X (dost. później)
    ⇒ odczyt zwraca zapisaną wartość
    - np. CPU B czyta X po kroku 3 przykładu
  - P_1 zapisuje X, P_2 zapisuje X
    ⇒ procesory widzą zapisy w tej samej kolejności
    - Kończy się tą samą końcową wartością X

Protokoły koherencji p. p.

- Operacje wykonywane przez p. p. w syst. Wieloprocesorowych dające koherencję
  - Migracja danych do lokalnych p. p.
    - Zmniejsza pasmo wspólnej pamięci
  - Powtarzanie wspólnych danych do odczytu
    - Redukuje „spór” o dostęp
- Snooping protocols („wścibskie” protokoły)
  - Każda p. p. śledzi odczyty/zapisy na szynie
- Protokoły oparte na katalogach
  - Pamięć i p. p. pamiętają stan współdzielenia bloków katalogu

Unieważnianie Snooping Protoc.

- P. p. otrzymuje wyłączny dostęp do bloku gdy jest zapisywany
  - Rozsyła przez szynę *invalidate message*
  - Późniejsze odczyty innych p. p. to nietrafienia
    - P. p. ściągają uaktualnioną wartość

Akcja CPU	Akcja na szynie	p. p. CPU A	p. p. CPU B	Pamięć
				0
CPU A czyta X	Nietrafienie X	0		0
CPU B czyta X	Nietrafienie X	0	0	0
CPU A pisze 1 w X	Unieważnienie X	1		0
CPU B czyta X	Nietrafienie X	1	1	1

Spójność pamięci

- Gdy zapisy są widziane przez inne procesory
  - “Widziane” oznacza, że odczyt daje zapisaną wartość
  - Nie jest to możliwe natychmiastowo
- Założenia
  - Zapis jest dokonany dopiero gdy wszystkie procesory go zobaczą
  - Procesor nie zamienia kolejnością zapisy i inne dostępy do danej
- Efekt
  - P zapisuje X, a następnie Y
    ⇒ procesory widzące nową Y widzą także nową X
  - Procesory mogą zmieniać kolejność odczytów, ale nie zapisów

Wielopoziomowe p. p.

Characteristic	ARM Cortex-A8	Intel Nehalem
L1 cache organization	Split instruction and data caches	Split instruction and data caches
L1 cache size	32 KiB each for instructions/data	32 KiB each for instructions/data per core
L1 cache associativity	4-way (I), 4-way (D) set associative	4-way (I), 8-way (D) set associative
L1 replacement	Random	Approximated LRU
L1 block size	64 bytes	64 bytes
L1 write policy	Write-back, Write-allocate(?)	Write-back, No-write-allocate
L1 hit time (load-use)	1 clock cycle	4 clock cycles, pipelined
L2 cache organization	Unified (instruction and data)	Unified (instruction and data) per core
L2 cache size	128 KiB to 1 MiB	256 KiB (0.25 MiB)
L2 cache associativity	8-way set associative	8-way set associative
L2 replacement	Random(?)	Approximated LRU
L2 block size	64 bytes	64 bytes
L2 write policy	Write-back, Write-allocate (?)	Write-back, Write-allocate
L2 hit time	11 clock cycles	10 clock cycles
L3 cache organization	–	Unified (instruction and data)
L3 cache size	–	8 MiB, shared
L3 cache associativity	–	16-way set associative
L3 replacement	–	Approximated LRU
L3 block size	–	64 bytes
L3 write policy	–	Write-back, Write-allocate
L3 hit time	–	35 clock cycles

Dwupoziomowa TLB

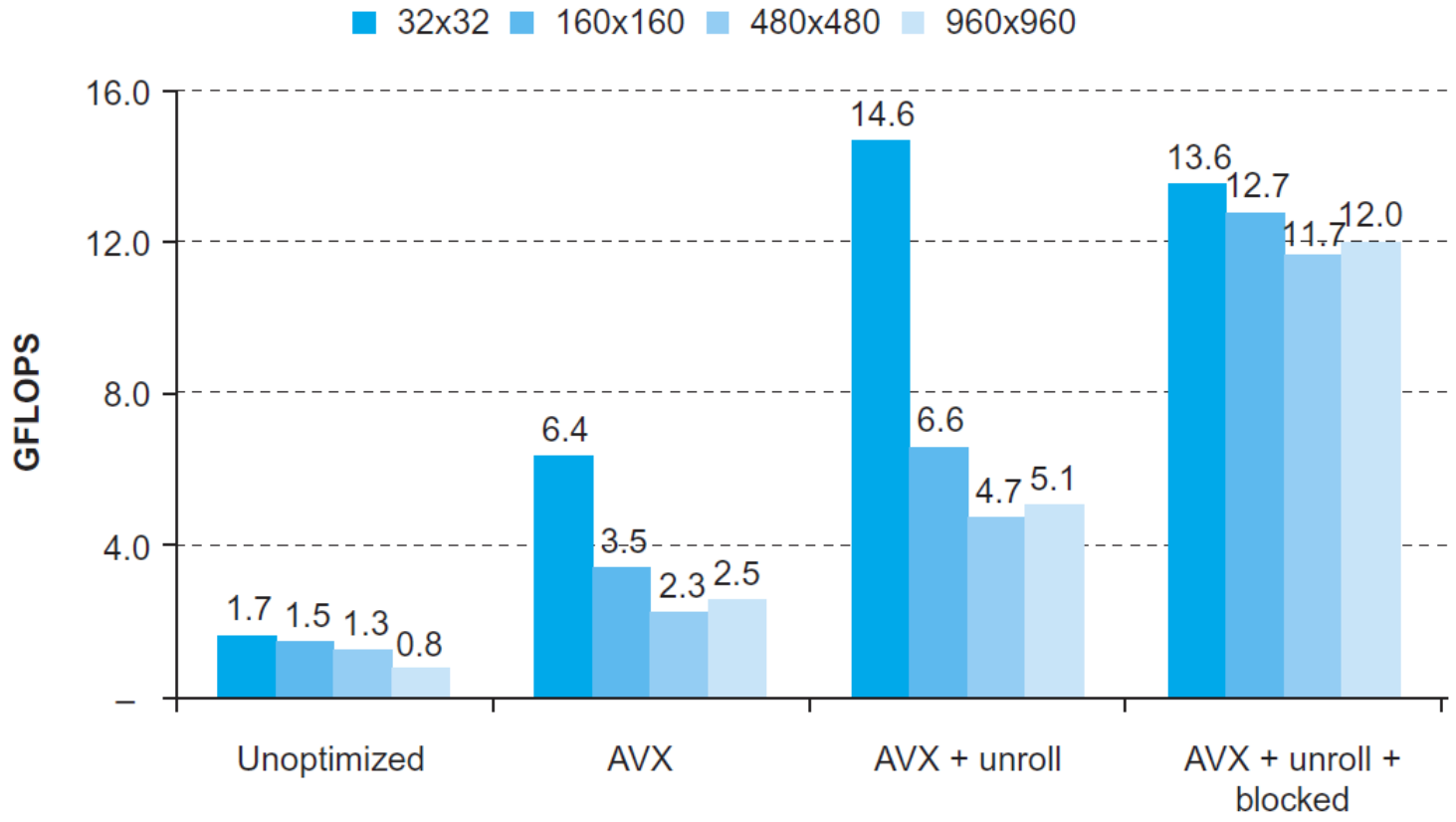
Characteristic	ARM Cortex-A8	Intel Core i7
Virtual address	32 bits	48 bits
Physical address	32 bits	44 bits
Page size	Variable: 4, 16, 64 KiB, 1, 16 MiB	Variable: 4 KiB, 2/4 MiB
TLB organization	1 TLB for instructions and 1 TLB for data Both TLBs are fully associative, with 32 entries, round robin replacement TLB misses handled in hardware	1 TLB for instructions and 1 TLB for data per core Both L1 TLBs are four-way set associative, LRU replacement L1 I-TLB has 128 entries for small pages, 7 per thread for large pages L1 D-TLB has 64 entries for small pages, 32 for large pages The L2 TLB is four-way set associative, LRU replacement The L2 TLB has 512 entries TLB misses handled in hardware

Wsparcie dla multiple issue

- Obydwa mają wieloczęściowe p. p. (multi-bank) pozwalające na wiele dostępów na cykl, o ile nie ma konfliktu w ramach banku
- Optymalizacja p. p. Core i7
  - W pierwszej kolejności daj żądane słowo
  - Nieblokująca p. p.
    - Trafienie pod nietrafieniem (Hit under miss)
    - Nietrafienie pod nietrafieniem (Miss under miss)
  - Wstępne ładowanie danych (Data prefetching)

DGEMM

- Połącz wieloblokowe p. p. i równoległość na poziomie podsłów



Pułapki

- Adresowanie: bajty a słowa
  - Przykład: 32-bajtowa p. p. z odwzorowaniem bezpośrednim, 4-bajtowe bloki
    - Bajt 36 odwzorowany na blok 1
    - Słowo 36 odwzorowane na blok 4
- Ignorowanie efektów systemu pamięci przy pisaniu lub generacji kodu
  - Przykład: analizowanie macierzy po kolumnach gdy ułożona wierszowo
  - Wielkie zakresy danych dają złą lokalność

Pułapki

- W wieloprocesorze z współdzieloną pamięcią podręczną L2 lub L3
  - Mniej asocjacyjności niż rdzeni daje nietrafienia z powodu konfliktów
  - Więcej rdzeni ⇒ konieczna większa asocjacyjność
- Używanie AMAT (śr. czas dostępu) dla oceny wydajności procesorów out-of-order
  - Ignoruje efekt nieblokowanych dostępów
  - Raczej oceniaj wydajność przez symulację

Pułapki

- Rozszerzanie pamięci przez segmenty
  - np. Intel 80286
  - Segment nie zawsze jest dostatecznie duży
  - Komplikuje arytmetykę adresów
- Implementowanie VMM na ISA nie projektowanej dla wirtualizacji
  - np. zasoby sprzętowe dostępne także przez instrukcje inne niż administratora
  - Albo rozszerz ISA, albo żądaj by OS gościa nie używał problemowych instrukcji

Uwagi końcowe

- Szybkie pamięci są małe, duże pamięci są wolne
  - Chcemy pamięci szybkich i wielkich ☹
  - Idea pamięci podręcznych (caching) daje taką iluzję ☺
- Zasada lokalności
  - Programy często używają mała część swojej pamięci
- Hierarchia pamięci
  - L1 cache ↔ L2 cache ↔ … ↔ pamięć DRAM ↔ dysk
- Właściwe projektowanie systemów pamięci dla wieloprocesorów jest krytycznie istotne